

Bitwise-Reproducible Reduction Across Cluster Reshape: A Compiler/Fabric ABI

Siddharth Patel, Aman Awasthi, Akshansh Chaurasia, and Rohit Singh

Abstract—Floating-point reductions executed across multi-accelerator systems are not bitwise reproducible under *cluster reshape*: when the number of processing elements G or the physical topology changes, the underlying collective’s reduction order changes, and so do the last bits of the result. Production collectives optimise throughput, not reproducibility, and therefore rerunning the same deterministic computation on a resized cluster yields bit-divergent weights, breaks scientific regression testing, and blocks audit-grade replay of machine-learning training. We argue that reduction order is an *interface* concern rather than a library concern, and treat it as a compiler/fabric Application Binary Interface: a size-indexed contract that fixes the operand binding at every internal node of a canonical full binary tree $\text{FBT}(N)$. Any fabric that enforces scoreboard-gated execution of this contract, and any compiler that emits an ABI-compliant instruction image, together yield a 64-bit result that is a pure function of N and the leaf vector L , independent of G , topology, assignment, mapping, and schedule. We instantiate the ABI as SPRS: a topology-parametric SystemVerilog network-on-chip with a four-opcode reduction ISA and a combinational flush-to-zero/round-to-nearest-even FADD; and a seven-stage compiler whose algorithmic configuration space spans $18 \times 10 \times 122 \approx 22,000$ assignment/mapping/refinement combinations, of which this campaign compiles 7,200 and validates 2,822 in RTL. Given the ABI, invariance is close to definitional — if every **COMPUTE** always adds the same predetermined pair of addresses, neither timing nor physical placement can reach the result. The contribution is therefore not that the property is surprising but that the discipline survives contact with a real compiler search space and real RTL at scale, which is not guaranteed. We state and argue two properties under six compiler obligations and nine fabric invariants: *cross-topology bitwise invariance* (Claim 1) and *fail-stop under assumption violation* (Property 1), connected by an explicit *mapping/topology orthogonality* theorem. On 40 instances spanning $N \in [8, 8192]$, $G \in [2, 4096]$, and six direct-network topologies (39 of the 40 hardware-validated), 2,822 RTL-validated configurations produce bit-identical roots with zero divergence, and an injected-fault campaign confirms the oracle is non-vacuous at 107/107 detection. We price the ABI structurally rather than empirically: its cost decomposes into a $\log_2 N$ dependence chain, a prohibition on locality-driven reassociation, leaf-generation serialisation, and root-gather fan-in. The RTL is cycle-accurately simulated across the suite. We further sketch *pinned-FBT*, a software-only export of the ABI discipline onto existing collective libraries (NCCL, MPI, oneCCL) that inherits the invariance property without adopting our fabric.

Index Terms—Deterministic reduction, collective communication, compiler/fabric co-design, IEEE 754 floating point, network-on-chip, bitwise reproducibility, all-reduce.

I. INTRODUCTION

The authors are with the Department of Electrical Engineering, Shiv Nadar University, India. E-mail: {sp839, aa634, ac198, rohit.singh}@snu.edu.in
Manuscript received August 12, 2026; revised X, XXXX.

DISTRIBUTED reductions are the connective tissue of modern parallel computation: gradient all-reduce in synchronous data-parallel training [1], [2], partial-sum aggregation in scientific codes [3], logits reduction in pipelined inference. In each case, a multiset of per-worker partial sums is collapsed into a single scalar or tensor through a collective whose performance has been the subject of more than a decade of aggressive engineering [4]–[7]. Performance, however, is not the same as determinism, and the two are in tension. Production implementations of all-reduce — ring, recursive halving-doubling, double-binary tree, hierarchical hybrid — select a reduction order that maximises throughput for the observed network topology and worker count [5], [8]. When either changes, so does the order.

IEEE 754 double-precision addition is non-associative: $(a + b) + c \neq a + (b + c)$ in the low-order bits whenever alignment shifts a non-zero mantissa below the round bit. The classical bound [9] gives, for two reduction orders σ and σ' of the same multiset,

$$|\sum_{\sigma} x_i - \sum_{\sigma'} x_i| \leq (N - 1)u \sum_i |x_i| + \mathcal{O}(u^2), \quad (1)$$

with u the unit in the last place. The classical statement bounds one summation order against the exact sum; applying it to the difference of two orders costs a factor of two by the triangle inequality, which we absorb into the constant since only the scaling matters here. Under cancellation this bound is catastrophically loose and the actual discrepancy is a function of the summation tree. The consequence is concrete. A training job rerun on a cluster resized from $G=128$ to $G=256$ GPUs — identical seed, identical dataset, identical code — produces bit-divergent weights by the end of the first iteration. The divergence propagates through every subsequent step and compounds. Bit-for-bit regression testing of scientific codes fails under the same mechanism: a code that ran to acceptance on a development cluster produces different last bits on the production cluster, and the reviewer cannot distinguish a new bug from the reshape artefact. In regulated deployments — medical-device inference, aerospace controls, financial model replay — the inability to reproduce a computation bitwise across hardware generations is increasingly disqualifying.

A. Two Sources of Non-Determinism, One Bad Outcome

The cluster-reshape non-determinism we target has two distinct sources, which current software tools confuse.

Source 1 — order-dependence of the reducer. The collective chooses a reduction tree or ring shape as a function of the runtime environment: the number of ranks G , the

inter-node bandwidth matrix, and in some implementations (NCCL, RCCL) a per-run bandwidth probe that selects the algorithm dynamically. Even with all randomness suppressed, the algorithm itself is a function of $(G, \text{topology})$, not of the input.

Source 2 — reassociation inside the reducer. Within a chosen algorithm, performance-critical implementations reorder partial sums to hide latency — overlapping pipeline segments, exploiting SIMD lanes, merging small messages. Each micro-optimisation is an implicit reassociation, and each changes the last bit.

Existing reproducibility tools address at most one of these sources. ReproBLAS [10] and ExBLAS [11] eliminate Source 2 within a single node via order-independent binned accumulation; they do not constrain the inter-node schedule and therefore inherit Source 1 from whatever collective composes them. The “deterministic” modes of production collectives (NCCL_ALGO=Tree with a single channel; Horovod-det; oneCCL-det) pin the algorithm within a fixed $(G, \text{topology})$ configuration; reshape the cluster and the tree shape changes by design. Vendor-deterministic accelerators [12], [13] guarantee per-chip reproducibility and inherit the vendor library’s cross-chip non-reshape guarantee — effectively, a stronger form of the fixed-configuration mode. Compensated-summation techniques [14], [15] address Source 2 at the per-reduction level but carry no statement about inter-worker order.

No existing approach guarantees that two executions of the same reduction on the same leaf values yield bit-identical results when the cluster size, topology, or schedule has changed. This is the gap we close.

B. Position: Reduction Order Is an Interface Concern

We take the position that bitwise reproducibility under cluster reshape cannot be achieved inside any single layer of the conventional software stack, and must instead be moved into the *interface* between the collective-scheduling compiler and the underlying fabric. A library-level flag cannot forbid Source 1, because the library has no authority over whether a different cluster size will be launched tomorrow. A fabric-level guarantee alone cannot forbid Source 2, because the fabric cannot see the reduction tree a scheduler intends to use. The discipline has to be a contract shared by both.

Concretely, let the operation be:

$$R = x_0 \oplus x_1 \oplus \dots \oplus x_{N-1}, \quad \oplus \triangleq \text{IEEE-754 FP64 ADD.} \quad (2)$$

We define a size-indexed contract: for every internal node v of a canonical full binary tree $\text{FBT}(N)$, the compiler emits exactly one compute instruction whose operand pair is $(\text{left}(v), \text{right}(v))$, and the fabric’s arithmetic unit reads those operands at a single clock edge after a scoreboard gate confirms both have been written. The contract is indexed only by N ; it says nothing about which processing element hosts which node, how packets are routed, or when any instruction fires. These are left free precisely so that schedulers may continue to optimise performance without touching the contract.

Under this discipline, the observed 64-bit root result is a pure function of N and the leaf vector L , irrespective of $(G, \text{topology})$, assignment α , mapping μ , refinement r , or timing. We call this property *fabric self-consistency under cluster reshape*. To be clear, we do not claim that this result agrees with any external reducer’s output — that would be a different (stronger) claim, requiring a shared canonical form. We claim that any two executions of our system on the same L agree bit-for-bit, under any reshape within a declared family $\mathcal{T}$ of flat direct-network topologies.

C. Contributions

This paper makes three contributions, supported throughout by runtime assertions in the compiler, dynamic SystemVerilog assertions in the fabric, and mutation testing. We use “mechanised” nowhere in its formal-methods sense: the invariance argument is by code inspection and dynamic checking, not machine-checked proof (§III-I).

C1 — Reduction order as a compiler/fabric ABI, with a structural invariance argument. We claim the framing, not the difficulty: once operand binding is a function of tree-node identity alone, invariance follows by unwinding the definitions, and we say so rather than dressing it as a discovery. We define the ABI as post-order operand binding over $\text{FBT}(N)$ together with a canonical leaf enumeration, and we state two properties that follow from it. *Claim 1 (cross-topology bitwise invariance)* establishes that, under six compiler obligations and nine fabric invariants, the 64-bit root result is a pure function of (N, L) . *Property 1 (fail-stop fabric)* establishes that every violation of an assumption produces either a correct result or a localised `P_ERROR` with diagnostic context, never a silently wrong output. The two are connected by an explicit *mapping/topology orthogonality* theorem. The argument proceeds by code inspection plus dynamic assertion, not mechanised proof; we discuss a path to mechanisation (§III-I).

C2 — A compiler and fabric that jointly realise the ABI. The SPRS compiler translates $(N, G, \text{topology})$ into a per-PE instruction image through seven stages: a composite lower-bound Ω (S0); $\text{FBT}(N)$ instantiation (S1); task-to-worker *assignment* selected from eighteen algorithms spanning list-scheduling, graph partitioning, tree dynamic programming, and critical-path variants (S2); a load-bearing *connectivity repair* pass that restores the ABI prerequisite after any S2 algorithm that fractures the partition (S2.5); worker-to-physical-PE *mapping* from ten algorithms (S3); *refinement* from twelve base methods (eleven refiners plus None), chainable to $12 + 11 \times 10 = 122$ configurations — each method alone, plus every ordered pair of two *distinct* refiners, since `refine_phased` applies at most two and never repeats one (S4); memory enforcement against 512-entry IMEM/DMEM budgets (S5); and code emission with cycle-accurate NOP padding (S6). The fabric is a topology-parametric SystemVerilog NoC with a four-opcode reduction ISA, a three-stage IF–DE–EX pipeline with two-level DMEM write-to-read forwarding, a scoreboard gate at DE, and a combinational IEEE-754 adder implementing flush-to-zero subnormals and round-to-nearest-even rounding.

Topology is a runtime configuration of adjacency and routing tables, not an RTL artefact.

C3 — Hardware validation at scale, and a structural pricing of the ABI. We evaluate on forty instances spanning $N \in \{8, \dots, 8192\}$, $G \in \{2, \dots, 4096\}$, and six topologies (linear, ring, mesh, torus, fat-tree, hypercube). The compiler’s configuration space spans $\approx 22,000$ points; the campaign compiles 7,200 of them and validates every feasible result with cycle-accurate Vivado xsim. The headline result is that all 2,822 RTL-validated configurations produce bit-identical roots at each N — across reshapes as large as $G=4$ on a linear chain to $G=128$ on a torus at $N=128$. We price the ABI structurally, in four components (tree-depth critical path, operand-binding locality, GCI serialisation, root-gather fan-in), rather than against a non-deterministic baseline we did not build. We report zero silent divergence across all configurations, and a detection rate of 107 of 107 mutants reaching a verdict, under two classes of injected mutations (address-substitution and leaf-corruption), confirming the oracle harness is non-vacuous.

D. Non-Goals

To forestall mis-scoping, we state explicitly what this paper does *not* claim.

Interoperability. SPRS results are self-consistent under reshape for a fixed SPRS/fabric pair. They do not match NCCL, RCCL, ReproBLAS, ExBLAS, or any external reducer whose canonical order differs. Interoperability requires either a thin input-permutation adapter (at zero runtime cost, trading the cross-reshape guarantee) or a shared canonical form — a direction we sketch in §VIII.

Strict IEEE 754. The FADD unit implements FTZ subnormals on input and output, RNE rounding only, a deterministic but non-canonical qNaN propagation rule, and a deterministic signed-zero rule that returns $+0$ on cancellation. These choices are documented (§III-G) and strictly narrower than IEEE 754-2019 [16]; a subnormal-aware alternative inherits Claim 1 by construction.

Silicon scalability. The fabric is a research prototype characterised by cycle-accurate RTL simulation. We make no claim of timing closure, area, power, or frequency at production-die scale, and we do not benchmark against production all-reduce libraries running on NVLink/NVSwitch silicon.

Novel component algorithms. Our eighteen assignment algorithms, ten mappings, twelve refinements, dimension-ordered routing, virtual-channel escape discipline, and scoreboarded pipeline are all individually due to prior work [17]–[24]. Our contribution is not a new component but the interface observation: that *if* compiler and fabric agree on a size-indexed reduction tree as a shared contract, *then* any choice among those prior-art components inherits cross-topology invariance by composition. The entire compiler variation space is structurally orthogonal to the ABI; the tournament optimises over a space incapable of changing the bit result.

E. Roadmap

Section II fixes notation for IEEE 754 non-associativity and the threat model, then tabulates prior reproducibility

approaches. Section III is the paper’s theoretical core: the canonical tree, the ABI definition, the compiler-side and fabric-side assumption lists, Claim 1, Property 1, and the orthogonality proposition. Section IV describes the fabric — ISA, pipeline, write-arbitration, fail-stop discipline, and resource envelope. Section V describes the compiler, with particular emphasis on the two load-bearing passes (connectivity repair, DMEM allocation) that construct the compiler-side obligations rather than assume them. Section VI describes the evaluation methodology (instance suite, tournament protocol, mutation testing, metric convention). Section VII presents the cross-topology bitwise invariance result, the mutation-detection campaign that shows the oracle is non-vacuous, and a structural pricing of the ABI together with the measured distance to Ω . Section VIII discusses scope, generalisation, and — most importantly — how the ABI discipline can be exported onto production collective libraries as *pinned-FBT*. Section IX covers related work. Sections X and XI state limitations and conclude. Appendices A–G provide the ISA/package specification, the full algorithm catalogue and winner distribution, a worked invariance example, per-instance coverage, and a reproducibility checklist.

II. BACKGROUND AND THREAT MODEL

A. IEEE 754 Non-Associativity

Let $x_0, x_1, \dots, x_{N-1}$ be N binary64 values and let σ be a summation order induced by a binary reduction tree τ . Denote by $\Sigma_\tau(x)$ the 64-bit result of evaluating τ with RNE rounding at every internal node. For two trees τ, τ' , the worst-case componentwise bound (1) is loose in the generic case but tight in the presence of cancellation: when $\sum_i x_i = 0$ by exact real arithmetic but intermediate partial sums differ in magnitude, the last bits of Σ_τ and $\Sigma_{\tau'}$ can differ by an exponent-magnitude of the largest partial sum. The practical consequence for reductions of noisy tensors is that two trees executing on identical x will, with overwhelming probability, produce different last bits of Σ .

Production collectives choose τ as a function of their schedule. Ring all-reduce [25] orders along a physical worker ring; a recursive halving–doubling all-reduce [8] orders by a power-of-two recursive pattern; double-binary-tree all-reduce [26] uses two interleaved binary trees. Runtime-hybrid collectives [5], [27] select an algorithm per call, sometimes per message size, based on bandwidth probes. In each case τ is a function of $(G, \text{topology}, \text{bandwidth probe})$. Reshaping the cluster reshapes τ and therefore Σ_τ .

B. Threat Model

We target a specific, precisely stated reproducibility property.

Definition 1 (Cross-topology bitwise reproducibility). Let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector. For every $(G, \text{topology}) \in \mathcal{T}$ (the declared family of supported direct networks), every valid SPRS compilation, and every valid runtime execution of the fabric, the observed 64-bit root result is the same sequence of bits.

The family $\mathcal{T}$ contains flat direct networks whose adjacency can be realised by a configuration of the per-router adjacency and routing tables: linear, ring, 2D mesh, 2D torus, hypercube, and fat-tree. The property is *structural* — it is a statement about what the fabric–compiler pair can be proven to guarantee, not an empirical average over executions.

Out of scope: (i) agreement between SPRS and external reducers at different canonical orders; (ii) tolerance to single-event upsets or transient hardware faults; (iii) cross-fabric portability of bit patterns between fabrics with different FADD microarchitectures; (iv) adversarial traffic patterns affecting cycle counts (these cannot affect the bit result; head-of-line behaviour is discussed in §IV-J).

C. Prior Reproducibility Approaches

Table I classifies prior reproducibility approaches along two axes: which source of non-determinism they eliminate, and at what scope. Single-node techniques (binned accumulation, compensated summation) target Source 2 but are silent on Source 1. Fixed-configuration techniques (NCCL-det, Horovod-det) pin Source 1 within one cluster size but fail under reshape. No prior approach addresses both sources at cluster scale. SPRS occupies the previously empty cell.

Adjacent lines of work motivate our approach without providing the cross-reshape guarantee. Compilers for collectives — TACCL [7], SCCL [6], BLINK [28] — synthesise performance-optimised schedules but treat reproducibility as out of scope. In-network reductions — SHARP [29] on InfiniBand and P4-programmable switches [30] — pin operand orders at the switch, a fabric-level insight we share; neither frames the guarantee as a size-indexed ABI, nor demonstrates cross-topology invariance.

III. THE REDUCTION ABI AND THE INVARIANCE CLAIM

This section is the paper’s theoretical core. We define the canonical reduction tree (§III-A), state the ABI as a pair of binding disciplines (§III-B), enumerate the fifteen assumptions on which the argument rests, cleanly partitioned into compiler obligations and fabric invariants (§III-C), and state the two properties that follow: cross-topology bitwise invariance (Claim 1) and fail-stop-under-violation (Property 1). The supporting arguments are given in §III-E. A key corollary — that the mapping and topology choices are *orthogonal* to the bit result — is stated and proven as an explicit mini-theorem in §III-F. We close with the FADD policy (§III-G), edge cases (§III-H), and a discussion of runtime checks and a path to mechanisation (§III-I).

A. The Canonical Tree

Definition 2 (Canonical full binary tree). For $N \geq 1$, the *canonical full binary tree* $\text{FBT}(N)$ is the heap-indexed binary tree with $2N - 1$ nodes: root at index 0, left child of i at $2i + 1$, right child at $2i + 2$. The set of internal nodes is $\{0, 1, \dots, N - 2\}$; the set of leaf nodes is $\{N - 1, \dots, 2N - 2\}$. Since $2N - 1$ is always odd, every internal node has exactly two children: the single-child edge case is eliminated at the tree level.

Definition 3 (Canonical leaf enumeration). Let $\mathcal{L}(N) \subset \{0, \dots, 2N - 2\}$ be the leaf set of $\text{FBT}(N)$. The canonical enumeration $\text{idx} : \mathcal{L}(N) \rightarrow \{0, \dots, N - 1\}$ assigns index k to the k -th smallest *heap index* in $\mathcal{L}(N)$.

This is worth stating precisely, because the obvious informal paraphrase is wrong. Ascending heap index and left-to-right *visual* order coincide exactly when N is a power of two. When it is not, $\text{FBT}(N)$ is depth-imbalanced: some leaves sit one level higher than the rest, and a higher leaf carries a *smaller* heap index than the deeper leaves to its left. For $N = 7$ the leaf set is $\{6, \dots, 12\}$; ascending heap index gives $\langle 6, 7, 8, 9, 10, 11, 12 \rangle$, whereas left-to-right traversal gives $\langle 7, 8, 9, 10, 11, 12, 6 \rangle$ — node 6 is a leaf at depth two and appears *last* visually. Seven of the sixteen leaf counts we evaluate (10, 50, 100, 200, 500, 1500, 3000) are of this shape, and at $N = 200$ the two enumerations produce root values differing by one ulp. The ABI is defined by heap index; an implementation using visual order is a *different* contract, not an equivalent restatement of this one.

The post-order traversal of $\text{FBT}(N)$ — left, right, node — is a pure function of N . We write π_N for the post-order sequence of internal nodes. For $N = 4$:

$$\pi_4 = \text{postorder}(\text{FBT}(4))_{\text{internal}} = \langle 1, 2, 0 \rangle, \quad (3)$$

i.e., internal node 1 (with children 3, 4) is evaluated first, then internal 2 (with children 5, 6), then root 0 (with children 1, 2). The canonical evaluation is

$$R_{\text{canonical}}(N, L) = \text{evaluate}(\text{FBT}(N), L, \text{fadd}_{\text{rtl}}), \quad (4)$$

where L is the leaf-value vector indexed by the canonical leaf enumeration of Definition 3 and fadd_{rtl} denotes the combinational semantics of our FP64 adder (§III-G). Equation (4) is the *target* of every SPRS execution on input (N, L) .

The canonical leaf enumeration idx of Definition 3 is likewise a pure function of N , and is the referent for every use of “leaf index” below.

Figure 1 illustrates $\text{FBT}(8)$ with its heap indexing, internal-node post-order, and leaf enumeration. The compiler constructs $\text{FBT}(N)$ by the `CBTree` class at `sprspr_core.py:380–482`; the class name is historical, the implementation constructs a full binary tree.

B. The ABI: Operand Binding and Leaf Binding

The ABI is a contract between compiler and fabric comprising two disciplines.

Definition 4 (Operand binding). For every internal node v of $\text{FBT}(N)$, the compiler emits exactly one `COMPUTE` instruction on the PE hosting v , with

$$\text{addr_a} = \text{dmem}(\text{left}(v)), \quad (5)$$

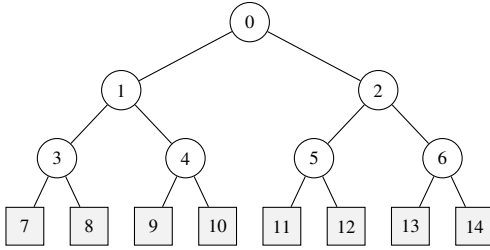
$$\text{addr_b} = \text{dmem}(\text{right}(v)), \quad (6)$$

where $\text{dmem}(\cdot)$ is a per-PE DMEM address assigned by the allocator. The binding is a pure function of v and depends on nothing else.

TABLE I
REPRODUCIBILITY APPROACHES, CLASSIFIED BY WHAT THEY FIX AND WHERE THEY FIX IT. SOURCE 1 = ALGORITHM SELECTION AS A FUNCTION OF $(G, \text{TOPOLOGY})$; SOURCE 2 = REASSOCIATION INSIDE THE ALGORITHM.

Approach	Order-fixing mechanism	Scope	Source 1	Source 2
ReproBLAS [10]	Binned accumulation	Single node	—	✓
ExBLAS [11]	Binned + long-accumulator	Single node	—	✓
NCCL-det, RCCL-det, oneCCL-det [5]	Fixed algorithm + single channel	One fixed $(G, \text{topology})$	✓*	✓
Horovod-det [1]	Algorithm pinning	One fixed $(G, \text{topology})$	✓*	✓
Kahan / pairwise / compensated [9], [14]	Compensated arithmetic	Single node	—	✓
Deterministic accelerator [12]	Per-chip deterministic execution	Single chip	—	✓
SPRS (this paper)	Size-indexed FBT(N) ABI	All $(G, \text{topology}) \in \mathcal{T}$	✓	✓

* Only within a fixed $(G, \text{topology})$; not preserved under reshape.



$\pi_g^{\text{internal}} = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$
leaf enum: $7 \mapsto 0, 8 \mapsto 1, \dots, 14 \mapsto 7$

Fig. 1. The canonical tree $\text{FBT}(8)$. Circles are internal nodes (heap indices 0–6); rounded squares are leaves (heap indices 7–14). The internal-node post-order $\pi_g^{\text{internal}} = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$ and the leaf enumeration idx are both pure functions of N and constitute the ABI's structural referents.

Definition 5 (Leaf binding). For every leaf $k \in \text{leaves}(\text{FBT}(N))$, the `GENERATE` instruction on the PE hosting k writes, into the DMEM slot bound to k , the 64-bit value $L[\text{idx}(k)]$, where idx is the canonical leaf enumeration of Definition 3.

The fabric's obligation is to execute the two disciplines symmetrically: `COMPUTE` reads the two DMEM slots at a single clock edge, scoreboard-gated, and writes $\text{fadd}_{\text{rtl}}(\text{value}_a, \text{value}_b)$ to the destination slot; `GENERATE` delivers $L[\text{idx}(k)]$ into its destination slot with the scoreboard set. Critically, neither discipline references G , topology, assignment α , mapping μ , refinement r , or timing. These are left free — and are the object of the tournament's optimisation in §VI.

C. Assumptions

We partition the fifteen assumptions on which the invariance argument rests into *compiler obligations* (C-*, six items; the compiler or user must discharge them), and *fabric invariants* (F-*, nine items; derivable by inspection of the RTL).

Compiler obligations:

C- α (Compilation determinism).

The assignment, mapping, and refinement stages produce outputs that are pure functions of their inputs and the random seed. All stochastic components (QAP-SA, ALNS, Tabu, Spectral initialisation) are seeded at a fixed value; Python 3.7+ insertion-ordered dictionaries eliminate the last source of hash-based nondeterminism.

C-conn (Partition connectedness).

For every PE p , the nodes assigned to p by α form a connected subtree of $\text{FBT}(N)$. Established by the S2.5 *connectivity repair* pass (§V-B); a load-bearing invariant without which the binding discipline fails.

C-sw (Single-writer allocation for remote destinations).

Partition each PE's DMEM into *receive buffers*, the addresses named as `target_addr` by some `SEND` from another PE, and *local temporaries*, the rest. For every PE p and every receive buffer $a \neq 0$, the emitted image contains exactly one write to (p, a) , and that write is remote. Local temporaries are exempt: the allocator reuses them (§V-C), and their safety comes from in-order issue rather than from single assignment — see the remark below. The two classes are disjoint by construction.

C-bind (Operand binding, Def. 4).

Every `COMPUTE` for internal node v binds the operand pair by definition. Verified by a post-generation assertion (§III-I).

C-leaf (Leaf binding, Def. 5).

Every `GENERATE` for leaf k delivers $L[\text{idx}(k)]$ into the DMEM slot bound to k . Codified by the testbench emitter (`emit_sv_testbench`, `sprcs_core.py:5526--5544`).

C-route (Routing-table soundness).

The emitted routing tables encode loop-free, destination-correct paths on the declared topology. Satisfied by construction for each topology family and checked acyclic by a channel-dependency pass (`validate_routing.py`).

Remark (Why address reuse does not break the gate). The allocator of §V-C returns a child's slot to a free list once the parent has consumed it, so a local temporary can be written more than once per program. Read against a scoreboard bit that is never cleared, that looks unsafe: a second writer would

find the bit already set, and a reader could not distinguish the new value from the old.

It is safe, and the reason is worth stating because it is what the two invariants above are really carving out. A recycled slot has exactly one remaining reader at the moment it is reused — the very `COMPUTE` that overwrites it — and on an in-order pipeline that instruction reads its operands in `DE` strictly before it writes back in `EX`. The ordering is supplied by program order on a single PE, not by the scoreboard. The scoreboard exists for the one case program order cannot cover: a value written by a *different* PE, whose arrival time is a function of routing and contention. Those destinations are precisely the receive buffers, and those are never recycled.

The separation is structural rather than incidental. The allocator assigns receive buffers in a second pass, from fresh addresses above the local high-water mark, and never from the free list. We verified the disjointness directly on the emitted images: across instances spanning $N \in [32, 2048]$ and $G \in [4, 128]$, the set of recycled addresses and the set of remotely-written addresses do not intersect in a single case.

Fabric invariants:

F-id (Identity slot).

`DMEM[0]` is the identity element of the selected ALU operation; `scoreboard[0]` is set to 1 at reset and never cleared until the next program start.

F-sb-mono (Scoreboard monotonicity).

For every receive buffer a , `scoreboard[a]` transitions $0 \rightarrow 1$ at most once per program, at the clock edge of the unique remote write to a , and stays set until the next program-level reset. The bit is never cleared mid-program, so the gate is monotone and cannot oscillate.

F-sb-gate (Scoreboard gating).

<code>COMPUTE</code>	stalls	<code>DE</code> → <code>EX</code>	until
both	<code>scoreboard[addr_a]</code>		and
	<code>scoreboard[addr_b]</code>		are 1.

F-fw (DMEM write-to-read forwarding).

A `DMEM` read at cycle t returns the value committed by the unique write to that address at cycle $\leq t$, even across the adjacent-cycle race. Realised by a two-level address-match forwarding network.

F-fadd (FADD purity).

`fp64_add.sv` is combinational and stateless (zero `always_ff`); its output is a pure function of the two 64-bit inputs sampled at the `EX` clock edge.

F-credit (Credit conservation).

For every link and every interval, `#credit_returns` $\leq$ `#buffer_reads` with a bounded wait. Realised by counter-serialised credit pulses that eliminate a multi-VC credit-loss bug present in an earlier design.

F-misroute (Misroute drop).

Packets arriving at a PE whose `dest_gpu_id` header field does not match the local PE are dropped and flagged (`piu_misroute`), not silently consumed. Converts a `C-route` violation from silent corruption into a liveness failure detected by `F-watchdog`.

F-arb (Write arbitration).

The `DMEM` write arbiter is strict-priority `PIU > GCI >`

FSM, with a `GCI` retry latch that re-asserts the `GCI` write until accepted; no write is silently dropped.

F-watchdog (Liveness guard).

A stall exceeding `WATCHDOG_MAX` = 20,000 cycles or an `EX` stall exceeding `TIMEOUT_MAX` = 10,000 cycles raises `P_ERROR`; invalid opcodes raise `P_ERROR` immediately.

Remark. The partition is not cosmetic. The `C-*` obligations can be checked by a compiler-side post-condition on the emitted `IMEM`, independently of any fabric execution — they are compile-time verifiable. The `F-*` invariants are properties of the fabric’s behaviour that can be established once by RTL inspection and SystemVerilog assertions, independently of any particular compilation. Separating the two means that a failure of Claim 1 can be diagnosed precisely: either the compiler violated an obligation (a bug in `SPRS`) or the fabric violated an invariant (a bug in the RTL), but not both.

D. Claim 1 and the Fail-Stop Property

Claim 1 (Cross-topology bitwise invariance). *Let $N \in \mathbb{N}$ and let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector. Let $\mathcal{T}$ be the family of flat direct-network topologies realisable by adjacency tables consistent with `noc_system.sv`. Under `C- $\alpha$` , `C-conn`, `C-sw`, `C-bind`, `C-leaf`, `C-route` and `F-id`, `F-sb-mono`, `F-sb-gate`, `F-fw`, `F-fadd`, `F-credit`, `F-misroute`, `F-arb`, `F-watchdog`, for all $(G, \text{topology}) \in \mathcal{T}$, all valid assignments α , mappings μ , refinements r , and schedules s emitted by `build_schedule`, the observed 64-bit root `DMEM` value at program termination equals*

$$R_{\text{observed}}(N, L, G, \text{topology}, \alpha, \mu, r, s) = R_{\text{canonical}}(N, L). \quad (7)$$

Under the same assumptions, a strictly stronger *safety* property holds. Violating any of the fifteen assumptions does not cause silent corruption of the result; it causes either a correct result (if the violation is benign) or a fail-stop detected by the fabric.

Property 1 (Fail-stop fabric). *Under any violation of `C- $\alpha$` , `C-conn`, `C-sw`, `C-bind`, `C-leaf`, `C-route` or `F-id`, `F-sb-mono`, `F-sb-gate`, `F-fw`, `F-fadd`, `F-credit`, `F-misroute`, `F-arb`, `F-watchdog`, the fabric terminates either (a) with a correct root value equal to $R_{\text{canonical}}(N, L)$, or (b) in `P_ERROR` with a recorded `error_pc` and `error_state`, or (c) in `watchdog` timeout with the same diagnostic, never in a silently-wrong root result.*

We label these a *claim* and a *property* rather than a mechanised theorem because the supporting argument proceeds by RTL inspection, compiler post-condition checks, dynamic SystemVerilog assertions, and mutation testing, not by formal proof. We discuss a mechanisation path in §III-I; readers who insist on “theorem” should substitute accordingly.

E. Supporting Arguments

Figure 2 shows the dependency structure of the assumptions and the supporting arguments. We present four arguments, each resting on a specific subset of assumptions.

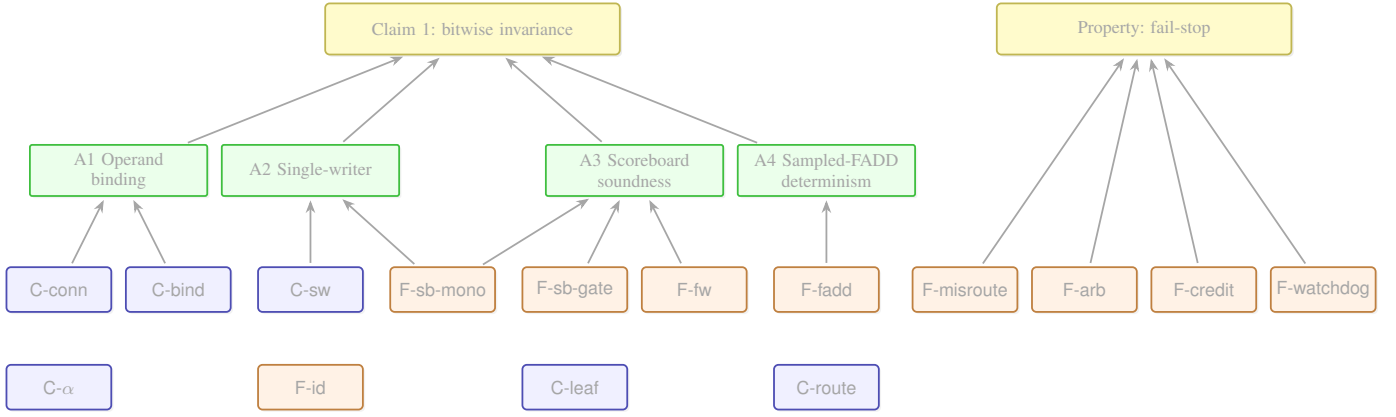


Fig. 2. Assumption dependency structure. Six compiler obligations (C-*, blue) and nine fabric invariants (F-*, orange) support four arguments (A1–A4, green) which establish Claim 1 (invariance) and Property 1 (fail-stop) in combination. Claim 1 rests on A1–A4; Property 1 rests additionally on the fail-stop invariants F-misroute, F-arb, F-credit, F-watchdog.

a) *Argument A1 (Operand binding, from C-bind, C-conn).*: `build_schedule` (`spr_core.py:5218--5222`) sets `child_addr` = [`addr(children[0])`, `addr(children[1])`]; the tree's `children(v)` method returns [`left(v)`, `right(v)`] by construction (`spr_core.py:404--408`). C-conn ensures that for every internal v , both `left(v)` and `right(v)` are either on the same PE as v (in which case their DMEM addresses are directly available) or reachable via RDMA-push into a receive-buffer slot pre-allocated on v 's PE. The operand binding therefore holds at the address level; under F-sb-gate (A3) and F-fw, the operand *values* read are the final values written.

b) *Argument A2 (Single-writer, from C-sw, F-sb-mono).*: The `DMEMAllocator` (§V-C) walks `su_postorder` and assigns each tree node a unique DMEM slot, recycling only after the node's unique parent has consumed it. Because `FBT(N)` is a tree, each node has exactly one parent and therefore exactly one consumer; SU-order guarantees that free-list reuse occurs after consume. This constructs C-sw. Under F-sb-mono, the scoreboard bit `scoreboard[a]` of each non-identity address transitions $0 \rightarrow 1$ exactly once in a program, at the clock edge of the unique write.

c) *Argument A3 (Scoreboard soundness, from F-sb-mono, F-sb-gate, F-fw).*: Under F-sb-mono, once `scoreboard[a]` is set, `DMEM[a]` contains the unique committed value at address a . Under F-sb-gate, the DE→EX transition of any `COMPUTE` instruction requires both operand scoreboard bits to be set. Thus at the EX clock edge both `DMEM[addr_a]` and `DMEM[addr_b]` contain the final values written by their unique writers. F-fw handles the BRAM read-first race: two cascaded forwarding registers catch the back-to-back-write-then-read case and the across-one-stall case, which together exhaust the possible races given the pipeline depth.

d) *Argument A4 (Sampled-FADD determinism, from F-fadd).*: At the EX-writeback clock edge, `fsm_dmem_wr` commits `faddrtl(dmem_a, dmem_b)` where the inputs are the stable values of A3. `fp64_add.sv` is combinational with

zero `always_ff` registers; its output is a pure function of its inputs.

e) *Composition.*: Every internal DMEM value at termination is `faddrtl` applied to the (left, right) children by Arguments A1, A3, A4, in post-order by data dependence, initial values coming from the C-leaf-enforced `GENERATE` writes. The root is therefore `evaluate(FBT(N), L, faddrtl) = Rcanonical(N, L)`, a pure function of (N, L) .

Remark (On Property 1). The fail-stop argument is more direct. A C-route violation routes a packet to the wrong PE, where F-misroute drops it; the destination's scoreboard bit is never set; `COMPUTE` stalls indefinitely; F-watchdog triggers `P_ERROR` after 20,000 cycles. A F-credit violation leads to link deadlock under finite buffering; again F-watchdog fires. A C-sw violation emits a duplicate write — caught at compile time by the allocator's invariant check — but even if it escaped, the second write's scoreboard transition violates F-sb-mono, which the fabric's SVA suite flags. No combination of plausible violations produces silent corruption.

F. The Mapping/Topology Orthogonality Theorem

Claim 1 asserts invariance over $(G, \text{topology}, \alpha, \mu, r, s)$. The (G, α) slice is where the substantive work lives: changing G changes the number of PEs, and changing α changes which PE hosts which tree node. The $(\text{topology}, \mu, r, s)$ slice is strictly less interesting — once the assignment is fixed, the topology, the worker-to-physical mapping, and the routing and timing only change *where* and *when* data flows, not the operand binding at any `COMPUTE`. We make this formal.

Proposition 1 (Mapping/topology orthogonality). *Fix N , L , assignment α , refinement r , and the fabric's FADD unit. For any two valid mappings μ_1, μ_2 and any two valid routing tables T_1, T_2 for (possibly different) topologies in $\mathcal{T}$, the observed root result satisfies*

$$R_{\text{observed}}(\mu_1, T_1) = R_{\text{observed}}(\mu_2, T_2) = R_{\text{canonical}}(N, L). \quad (8)$$

Structural argument. By inspection of `build_schedule` and `generate_instructions` (§V-E): the `COMPUTE` in-

structions' $(\text{addr_a}, \text{addr_b}, \text{dest})$ triples depend only on α and the DMEM allocator's output, neither of which reads μ or the routing tables. The SEND instructions' $(\text{dest_gpu}, \text{target_addr}, \text{link})$ triples depend on μ (through the `mapping.get(gpu, gpu)` indirection) and on the routing tables — but these only affect *where* data flows, not the $(\text{addr_a}, \text{addr_b})$ operand bindings at the receiving PE's COMPUTE. Under F-sb-gate, the receiving PE's COMPUTE reads the DMEM values by address, not by arrival time, arrival port, or arrival VC. Therefore R_{observed} is independent of (μ, T) . $\square$

Proposition 1 is the cleanest statement of what the ABI buys. The topology and the physical mapping — the two things that change most obviously under cluster reshape — are structurally orthogonal to the bit result. This is not a consequence to be validated empirically; it is a consequence of the code path that emits the instructions.

G. FADD Policy

Our FADD (`fp64_add.sv`, with a bit-exact Python mirror at `sprspr_core.py:702--867`) implements a *deterministic subset* of IEEE 754-2019 binary64 addition. We specify the subset fully:

- 1) *Input FTZ*. Subnormal inputs ($\text{exp} = 0, \text{frac} \neq 0$) are flushed to ± 0 preserving sign.
- 2) *RNE only*. The round-to-nearest-even predicate is `guard ^ (sticky v lsb)`. No other rounding mode is available; there is no latched rounding-mode register.
- 3) *Output FTZ*. Post-round subnormals are flushed to ± 0 preserving sign.
- 4) *Signed-zero cancellation*. The result of $x + (-x) = +0$ for all non-NaN x ; $(-0) + (-0) = +0$. The latter deviates from IEEE 754-2019 §6.3, which mandates -0 ; we document the deviation and note it is deterministic and inherits Claim 1.
- 5) *NaN propagation*. Output is a quieted qNaN whose sign equals `sign(a)`, exponent is `0x7FF`, bit 51 is forced to 1 (the quiet bit), and bits `[50:0]` copy the payload of a if a is NaN, else of b . Narrower than IEEE 754 (which allows any qNaN payload) but deterministic.
- 6) *Infinity arithmetic*. $\pm\infty + \pm\infty$ (same sign) $= \pm\infty$; $\pm\infty + \mp\infty$ returns the canonical qNaN `0x7FF8_0000_0000_0000`; $\pm\infty + \text{finite} = \pm\infty$.

We refer to (1)–(6) collectively as *FTZ+RNE semantics*. Invariance (Claim 1) depends only on `fadd_rtl` being a pure function of its inputs; it does not depend on any particular IEEE 754 feature.

Compatibility with reference implementations. FTZ+RNE diverges from strict IEEE 754 on three regimes: subnormal inputs, post-round subnormal outputs, and signed-zero cancellation of two negative zeros. Workloads whose reference implementation (NumPy, MATLAB, a reference BLAS) produces or depends on subnormals will see SPRS diverge from reference at the affected bits. This is a scope decision, not an oversight: a subnormal-aware alternative `alu_op` slot is implementable as a one-instruction ISA extension that inherits Claim 1 by

construction. We flag the extension for deployments requiring reference-library bit-parity.

H. Edge Cases

A few edge cases deserve explicit treatment.

$N = 1$. FBT(1) is a singleton (one leaf, zero internal nodes). The “result” is $L[0]$; the compiler emits one GENERATE and no COMPUTE. The argument is vacuously true.

$N = 0$. Disallowed at the compiler front end.

Single-child edge. By construction, FBT(N) with $2N - 1$ nodes has no single-child internal nodes. The identity slot `DMEM[0]` is reserved but unused in the present workload; it exists so that future relaxations (asymmetric trees, variable-arity reductions) can use an identity operand without changing the ISA.

Integer and bitwise reductions. Claim 1 requires only a deterministic binary operation; it does not require FADD specifically. Integer ADD, MAX, MIN, bitwise operators inherit the claim by extension via a different `alu_op` code. This is a structural extension, not an empirical generalisation.

I. Mechanisation and Runtime Checks

Our current strategy discharges A1–A4 through a combination of *compile-time* post-condition assertions, *simulation-time* SystemVerilog assertions, and *empirical* mutation testing.

a) Compile-time checks.: After `build_schedule` emits the IMEM, a 120-line verifier `validate_abi.py` checks C-bind, C-sw, C-leaf:

- For every internal v and its emitted COMPUTE, `addr_a = addr_map[pe(v)][left(v)]` and `addr_b = addr_map[pe(v)][right(v)]`.
- For every PE p , $|\{(a, \text{dest}) : \text{dest} = a\}| = |\text{addr_map}[p]|$ (single-writer).
- For every leaf k , the GENERATE writes `gci_buffer[local_idx(k)] = make_leaf_val(idx(k))`.

Compilation fails on any violation. A companion `validate_routing.py` builds the channel-dependency graph on the emitted routing tables and verifies acyclicity per topology, discharging C-route.

b) Simulation-time SVA.: The testbench `tb/sva/tb_sva_e2e.sv` asserts: no credit underflow or overflow per port; no packet arrival on an invalid (port, vc) pair; no `scoreboard_hazard` firing at a previously-set bit; `piu_misroute` never asserts on a correctly routed packet; the end-to-end 64-bit result matches the Python oracle.

c) Mutation testing.: To establish that the oracle harness is non-vacuous (i.e., would detect a real divergence were one to occur), we inject two mutation classes:

- *Address substitution*: in one randomly chosen COMPUTE, replace `addr_b` with the DMEM address of a different tree node. Breaks Argument A1 while preserving A2–A4.
- *Leaf corruption*: in one randomly chosen GENERATE, flip one mantissa bit.

We *do not* use operand-swap as a mutation: FADD is commutative at the bit level under FTZ+RNE (`fadd_rtl(a, b) =`

$\text{fadd}_{\text{rtl}}(b, a)$ for all (a, b) , so operand-swap is bit-identical and would inflate the false null-detection rate spuriously. Mutation detection rates are reported in Table V (§VII-B).

d) *Path to mechanisation.*: A bounded Alloy [31] model of the scheduler plus scoreboard (scope $N = 16, G = 4$, ~ 250 lines) would mechanise Arguments A1–A3 at finite scope. A HardCaml or Kami [32] embedding of `btree_fsm_fast.sv` and `fp64_add.sv` would mechanise A4. We report these as in-progress supplementary artefacts and do not claim them here.

IV. THE FABRIC: NOC, ISA, PIPELINE

This section describes the fabric that realises the F^* invariants. The guiding principle is that topology is a runtime configuration of adjacency and routing tables (§IV-B), the ISA is small enough to audit for ABI adherence (§IV-D), the scoreboard is the single point of synchronisation across four independent write agents (§IV-E), and every assumption violation terminates the program at a diagnostic state rather than producing silent corruption (§IV-F).

A. Design Goals

The fabric supports a cluster of up to `MAX_ROUTERS = 4096` PEs, in a single Verilog instantiation whose topology is determined by the adjacency and routing tables loaded at boot. Router radix is not fixed by the design: it is the per-instance parameter `PORTS_PER_RTR`, which the compiler emits and which ranges 3–66 across our suite (§IV-I). The reduction ISA has four live opcodes, one reserved for future barrier/reset semantics. All determinism-critical mechanisms — scoreboard, forwarding network, DMEM write arbiter, credit accounting — are simple enough to fit in a two-page RTL inspection.

B. Topology as a Runtime Parameter

Two per-router configuration tables parameterise the topology.

The *adjacency table* (`adj_entry_t` in `noc_pkg.sv`) maps $(\text{router}, \text{port}) \rightarrow (\text{target_router}, \text{target_port}) \mid \text{ADJ_DISCONNECTED}$, set by the compiler via a write-enable interface before `program_start`. The *routing table* (`route_table` in `noc_router.sv`) maps $(\text{router}, \text{destination}) \rightarrow \text{output_port}$, also compiler-loaded. A single `noc_system` module with `N_NODES` and `N_ROUTERS` parameters supports the full family $\mathcal{T}$. Port 0 is reserved for the local network interface on every router — the one structural convention independent of topology.

The consequence is that *topology is data, not RTL*. Adding a new topology requires implementing a Python classmethod that emits adjacency and routing tables; no Verilog change is needed. The compiler at `spr_core.py:487--695` provides six topology families (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube) with analytical shortest-path computations at $O(1)$ per pair; a BFS fallback handles custom topologies at $O(G)$ per source.

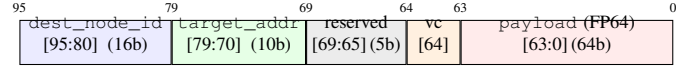


Fig. 3. 96-bit single-flit packet format. Bit [64] is the VC-steering bit: VC selection is a pure function of the packet header, independent of the 64-bit payload. Routing latency is therefore value-agnostic, which is the hidden determinism-relevant reason the same-fabric cycle count does not depend on input value distributions.

TABLE II
THE FOUR-OPCODE REDUCTION ISA. `OP_RESERVED` IS RESERVED FOR A FUTURE BARRIER/RESET_SB INSTRUCTION ENABLING MULTI-REDUCTION PROGRAMS. OPCODES $\geq 4'h4$ (OTHER THAN THE RESERVED SLOT) TRAP TO `P_ERROR`.

Mnemonic	Code	Effect
NOP	4'h0	Consume one slot.
GENERATE	4'h1	Fetch a leaf into <code>DMEM[dest]</code> via the GCI interface; set <code>scoreboard[dest]</code> .
COMPUTE	4'h2	Read <code>DMEM[addr_a]</code> , <code>DMEM[addr_b]</code> (scoreboard-gated), write <code>alu_op(a,b)</code> to <code>DMEM[dest]</code> , set <code>scoreboard[dest]</code> .
SEND	4'h3	Enqueue an RDMA-push packet with payload <code>DMEM[src]</code> to $(\text{dest_gpu}, \text{target_addr})$ on link.
<code>OP_RESERVED</code>	4'h4	Reserved for future BARRIER/RESET_SB.

C. Packet Format

The 96-bit single-flit packet format (Fig. 3) carries exactly one FP64 value per packet:

$$\underbrace{\text{dest_node_id}}_{16\text{ b}} \parallel \underbrace{\text{target_addr}}_{10\text{ b}} \parallel \underbrace{\text{reserved}}_{5\text{ b}} \parallel \underbrace{\text{vc_select}}_{1\text{ b}} \parallel \underbrace{\text{payload}}_{64\text{ b}} \quad (9)$$

Bit [64] is the VC-steering bit (the VC-select comparison in `noc_router.sv`). VC selection is a function of a *header* bit, not any payload bit. Were VC selection payload-dependent, contention and buffer occupancy would be value-dependent; cycle counts would be adversarially perturbable under value distributions. The scoreboard-gated ALU would still produce the correct FADD under the F^* invariants, but the overhead characterisation would no longer be value-agnostic. Header-only VC steering therefore plays a role in the empirical-repeatability argument even though it does not appear in Claim 1.

D. Reduction ISA

A four-bit opcode addresses sixteen slots; four are live (Table II), eleven are reserved for trap, one is earmarked.

`DMEM[0]` is reserved as the identity element of the active ALU operation; `scoreboard[0]` is initialised to 1 on reset (the DMEM write port in `btree_fsm_fast.sv`); the allocator reserves address 0 (`spr_core.py:5098`). This realises F-id and supports compiler emission of identity-preserving half-reductions.

E. Pipeline and DMEM Forwarding

Each PE hosts a three-stage pipeline:

- **IF**: registered IMEM read (1-cycle latency), PC management, combinational decode of opcode, aux, dest, addr fields.
- **DE**: scoreboard hazard check, DMEM read issue. Stalls if either scoreboard bit is 0 (**F-sb-gate**).
- **EX**: combinational ALU (**FADD** and friends), DMEM writeback or TX FIFO enqueue. Stalls on **SEND** back-pressure or DMEM write collision.

The DMEM read path uses a two-level write-to-read forwarding network (the forwarding network in `btree_fsm_fast.sv`), realising **F-fw**. Level 1 forwards a cycle- N write to a cycle- $(N+1)$ read of the same address — the back-to-back-COMPUTE case. Level 2 forwards across a single intervening stall, catching the case where the one-cycle forward expired before the read settled. The two levels together exhaust the possible races given the pipeline depth and the scoreboard's one-cycle gate.

The scoreboard is a `DMEM_DEPTH`-wide bit vector set whenever *any* of four write agents commits a write: PIU (from an arriving RDMA-push packet), GCI (from a completed leaf fetch), FSM COMPUTE writeback, or GENERATE (the FSM path that closes a GENERATE by committing its fetched value). **F-sb-mono** guarantees each bit transitions $0 \rightarrow 1$ at most once per program; the scoreboard is the single point where “data present at address a ” is decided, independently of which of the four agents delivered the data. This is what makes the scoreboard-gated reads structurally race-free (Fig. 4).

F. Fail-Stop Discipline

The fabric implements a three-way safety net that makes Property 1 provable rather than asserted.

PIU autonomous write and misroute drop. The PIU (the PIU block in `btree_fsm_fast.sv`) drains RX FIFOs on a priority basis. On each drained packet it examines bits [95:80] (`dest_gpu_id`). If equal to the local PE's id, it writes the payload to `DMEM[pkt[79:70]]` (`target_addr` from the header) and sets the scoreboard bit. If not equal — a routing-table bug, a corrupted packet, a torn adjacency — it drops the packet, returns credit, and asserts `piu_misroute` for one cycle. This realises **F-misroute**.

The consequence for determinism is stronger than a silent-failure assumption. A bad routing table causes the destination's scoreboard bit never to be set; the destination's dependent COMPUTE stalls forever; **F-watchdog** (below) fires and raises `P_ERROR`. A routing-table violation cannot produce a silently-wrong root result — the fabric converts the violation into a liveness failure detected and localised by the watchdog.

Watchdog + stall timeout + invalid-opcode trap. The fabric maintains three error detectors in the `P_RUN` state (the stall logic in `btree_fsm_fast.sv`):

- Watchdog: fires after `WATCHDOG_MAX = 20,000` cycles with no instruction retired.
- Stall timer: fires after `TIMEOUT_MAX = 10,000` cycles of continuous EX stall.
- Invalid-opcode trap: any opcode $> 4'h3$ (other than the reserved slot) triggers `P_ERROR` immediately.

Each raises `error_pc` and `error_state` for post-mortem diagnosis. The complete list of termination outcomes is therefore: (i) correct result; (ii) `P_ERROR` with diagnostic context; or (iii) watchdog-timeout with the same diagnostic. There is no silent-corruption outcome under any configuration we have exercised.

G. Write Arbitration and Retry Latches

The four DMEM write agents demand the same resource; their arbitration (Fig. 4, bottom) realises **F-arb**.

The arbiter is strict-priority `PIU > GCI > FSM` (the DMEM write arbiter in `btree_fsm_fast.sv`). PIU always wins — maintaining the RDMA-push semantic that incoming data is never blocked. The GCI write carries a retry latch (`gci_write_held` in `btree_fsm_fast.sv`): if GCI completion (`gci_done`) collides with a PIU write in the same cycle, the arbiter drops the GCI write but the retry latch captures the pending value and re-asserts `gci_dmem_wr` until the arbiter accepts it. The FSM COMPUTE writeback loses to both agents and is stalled via `ex_dmem_collision` when necessary.

This matters for determinism in a subtle way. Without the GCI retry latch, a PIU/GCI collision would silently drop the leaf value; the destination's scoreboard would never be set by GCI (PIU wins so the bit is set, but to the wrong value); the next COMPUTE would read a wrong leaf and produce a wrong root — violating A1 (sampled-read determinism) despite all other invariants holding. The retry latch is a narrow safeguard against a specific timing race; we name it explicitly because it is necessary-for-correctness, not merely nice-to-have.

H. Routing Discipline and Deadlock Freedom

The fabric provides `NUM_VCS = 2` virtual channels per port (`DEFAULT_VCS = 2`, `noc_pkg.sv`). Per-link credit-based flow control prevents buffer overflow: the `link_tx` module decrements a per-link credit counter on each transmission and increments it on each credit-return pulse, with counter conservation $\sum \text{pulses} = \sum \text{reads}$ (**F-credit**). Credit return is counter-serialised (the credit-return counter in `noc_router.sv`). This design fixes a multi-VC credit-loss bug in which a single-bit credit pulse collapsed several simultaneous reads into one return: the corrected implementation counts $\text{new_reads} = \sum_v \text{ibuf_ren}[p][v]$ and drains one pulse per cycle. This per-cycle read tally is a distinct register from the eight-bit per-link credit *balance*: the tally counts only simultaneous read events across VCs, so two bits suffice at `NUM_VCS = 2` even though the balance itself ranges to `BUF_DEPTH = 8`.

For cyclic topologies (ring, torus, hypercube), the compiler emits routing tables that satisfy a dimension-ordered (DOR) restriction with a VC-escape discipline [24]: packets route dimension-by-dimension, and packets crossing a wrap-around edge use VC1; non-wrap traffic may use either VC. This discharges **C-route**: the channel-dependency graph on the emitted routing tables is acyclic, and a 30-line Python check (`validate_routing.py`) verifies acyclicity for every compiled instance.

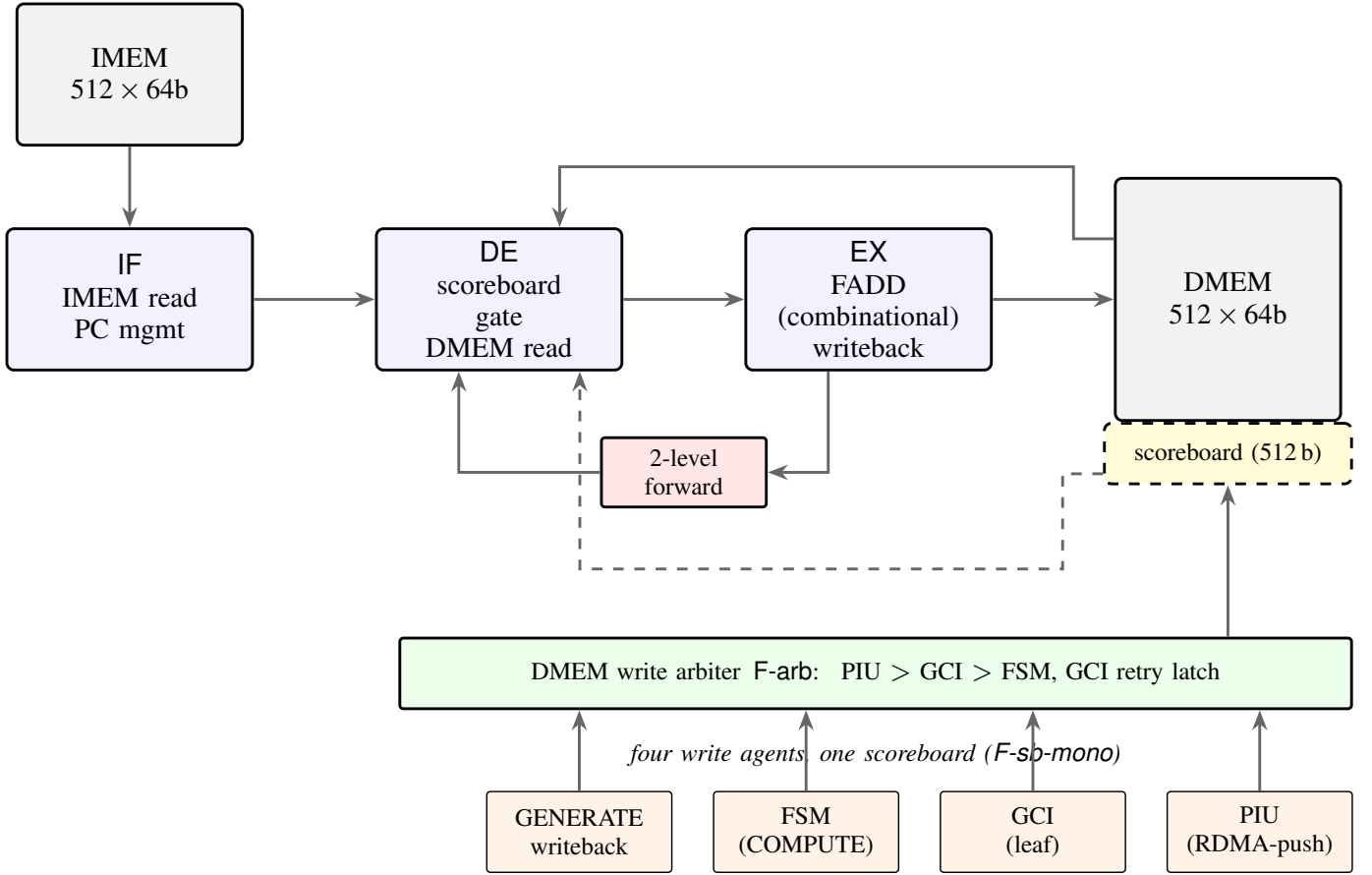


Fig. 4. PE microarchitecture. The three-stage IF-DE-EX pipeline is scoreboard-gated at DE; the scoreboard is a single source of truth for “data present at address a ” across four independent write agents (PIU, GCI, FSM COMPUTE writeback, GENERATE). A two-level write-to-read forwarding network (red) handles the BRAM read-first race for back-to-back writes and across-one-stall writes. The DMEM write arbiter (green) enforces the strict-priority $\text{PIU} > \text{GCI} > \text{FSM}$ discipline of F-arb; the GCI retry latch re-asserts the pending write until accepted. No write is silently dropped.

The router does not hardware-enforce the VC-escape discipline; correctness depends on C-route holding at compile time. We corroborate dynamically via SVA; a hardware-level dateline check (a one-register-per-link dateline flag) is a straightforward extension for production deployment.

Round-robin allocation at each output port advances only on a successful grant (the round-robin arbiter in `noc_router.sv`): `rr_ptr[i]` increments iff `obuf_wen[i]` this cycle. Without this guard, RR advance on empty cycles would allow unfairness; with it, every contending $(port, vc)$ pair is served within `NUM_CANDIDATES` cycles of becoming ready. This is a liveness property of the NoC.

I. Resource Envelope

Router cost is dominated by radix, and radix is a property of the topology, not a constant of the design. `MAX_PORTS` is declared in `noc_pkg.sv` but is never referenced by any module; the live quantity is `PORTS_PER_RTR`, which the compiler emits per instance. We read it back out of every generated testbench rather than assume it: Table III reports what the suite actually instantiates.

The span is 3–66 ports. Low-diameter mesh and torus routers stay at $P = 5$, hypercube reaches $P = 11$ at $G = 1024$, and fat-tree reaches $P = 66$ at $G = 4096$ (`maxp_mod_fat`,

TABLE III
MEASURED ROUTER RADIX ACROSS THE 40-INSTANCE SUITE, READ FROM THE PPR LOCALPARAM OF EACH EMITTED TESTBENCH. RADIX IS A TOPOLOGY PROPERTY; `MAX_PORTS` IN `noc_pkg.sv` IS VESTIGIAL AND ENFORCES NOTHING.

Topology	Radix rule	Radix P	$\lceil \log_2 P \rceil$
linear	2 + local	3	2
ring	2 + local	3	2
mesh	4 + local	5	3
torus	4 + local	5	3
hypercube	dim + 1	3–11	2–4
fat-tree	$\max(5, k+2)$	5–66	3–7

4,192 routers). Because the crossbar term is $O(P^2)$, a single fixed-radix estimate is not merely imprecise across topologies — it is wrong by more than an order of magnitude at the top of the range. Table IV therefore reports the envelope as a function of P , holding the route-table depth at $G = 4096$ so that the columns isolate the effect of radix alone.

A fat-tree router is $23\times$ a mesh router (598,912 bits against 26,208), and 70% of that is crossbar. Aggregated over the fabric the gap is starker still: ≈ 102 Mbit of router storage at $P = 5$ over 4096 routers, against ≈ 2394.3 Mbit for `maxp_mod_fat` — the regime where a flat crossbar stops

TABLE IV

PER-ROUTER RESOURCE ESTIMATE AS A FUNCTION OF RADIX P , AT A COMMON ROUTE-TABLE DEPTH $G = 4096$. A RESEARCH-PROTOTYPE ENVELOPE (STORAGE-EQUIVALENT BITS; THE CROSSBAR ROW PRICES MUX STORAGE-EQUIVALENT, NOT REGISTERS); NO POST-PLACE-AND-ROUTE DATA.

Component	Per-router bits at radix P		
	$P = 5$ mesh/torus	$P = 11$ hypercube	$P = 66$ fat-tree
Routing table ($G \lceil \log_2 P \rceil$)	12,288	16,384	28,672
VC ingress FIFOs ($P \cdot 2 \cdot 8 \cdot 96$)	7,680	16,896	101,376
Egress FIFOs ($P \cdot 8 \cdot 96$)	3,840	8,448	50,688
Crossbar ($P^2 \cdot 96$)	2,400	11,616	418,176
Per-router subtotal	26,208	53,344	598,912

being the right structure and a hierarchical or input-queued switch is the standard answer. We report the flat figure because it is what we simulated, not because it is what we would build at that radix.

Per-PE memory is independent of topology: IMEM 512×64 plus DMEM 512×64 is 65,536 bits per PE, ≈ 256 Mbit over $G = 4096$ PEs. This is the row Appendix A’s 64-bit instruction word sizes.

Two caveats bound the estimate. It counts storage-equivalent bits only — link drivers, clock distribution, arbiter logic, and I/O are excluded — and it is an RTL-level count, not a synthesis result. The critical combinational path is through `fp64_add` at EX (single cycle in our implementation); a production deployment would pipeline the FADD to 3–5 stages and migrate scoreboard release to ALU completion. Neither change affects the invariance claim — pipelined-but-still-deterministic FADD inherits Claim 1 by construction.

J. Known Limitations

To set the scope of the present implementation honestly:

Head-of-line blocking. The router is store-and-forward with a single FIFO per VC; under the tree-reduction traffic pattern, root-incident links queue up to $\lceil \log_2 G \rceil$ merge waves. At $G \geq 1024$ on ring and torus, absolute cycle counts slew because of this; the ratio against the non-deterministic baseline is more stable because both schedules share the bottleneck. Wormhole forwarding is a planned extension.

Single clock domain. No clock-domain crossings. Multi-chiplet deployment at $G = 4096$ would require asynchronous FIFOs on inter-chiplet links, changing credit round-trip latencies but not the bit result.

Credit-pulse counter width. At `NUM_VCS = 2` the counter is 2 bits, sufficient at `BUF_DEPTH = 8`; reviewer-flagged for extensions to `NUM_VCS > 4`. An SVA check bounds the counter.

Dynamic-only deadlock enforcement. The router does not hardware-enforce the VC-escape discipline; correctness depends on C-route. A hardware dateline is future work.

Soft-enforced single-writer. C-SW is discharged by a compile-time post-condition over the emitted IMEM, not by hardware refusal of a second write to an already-set scoreboard

slot. A bug that silently corrupted the allocator would be caught by our assertion harness but not by the fabric. A one-line RTL extension — “raise `P_ERROR` on a scoreboard-set transition from 1 to 1” — would harden this into a fabric-side guarantee; we plan it for the next revision.

V. THE SPRS COMPILER

This section describes how the compiler discharges the six C-* obligations. Two passes — connectivity repair (§V-B) and SU-ordered DMEM allocation (§V-C) — do the load-bearing work; they *construct* invariants rather than assume them. The remaining stages are standard heuristic ensembles, and their specific algorithmic choices matter for cycle counts but not for the bit result.

A. Compilation Pipeline

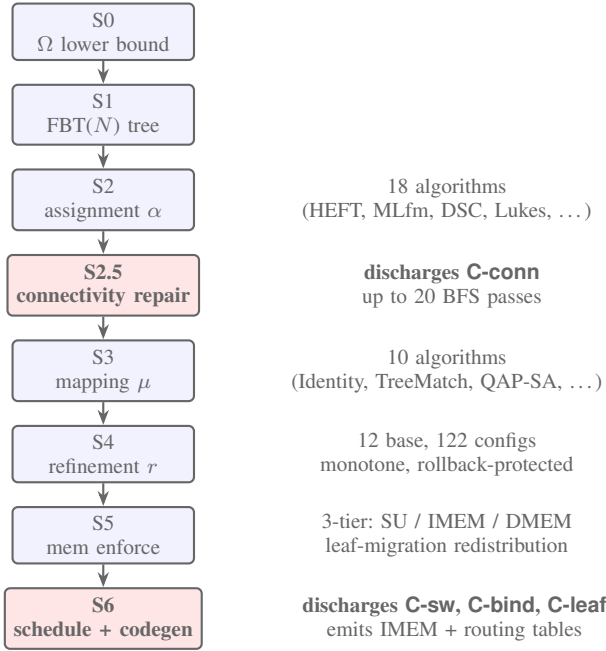
SPRS compiles a triple $(N, G, \text{topology})$ into a per-PE IMEM image plus per-router adjacency and routing tables via seven stages:

- S0** Composite lower bound Ω (`compute_lower_bound`) — secondary metric, see §VI-A.
- S1** FBT(N) instantiation (`CBTree.__init__`).
- S2** Assignment α : tree nodes $\rightarrow$ GPUs, selected from 18 algorithms (Appendix B, Tab. VIII).
- S2.5** *Connectivity repair* (`repair_connectivity`) — discharges C-conn.
- S3** Mapping μ : GPUs $\rightarrow$ physical PEs, from 10 algorithms.
- S4** Refinement r : 12 base methods, chainable to 122 configurations; rollback-protected composition (`refine_phased`) monotone in makespan.
- S5** Memory enforcement against IMEM/DMEM budgets of 512 (`enforce_memory`).
- S6** ISA emission with cycle-accurate NOP insertion and GCI-gap peephole compression (`build_schedule + generate_instructions`).

All three algorithmic search stages (S2, S3, S4) preserve the ABI by Argument A1 (§III-E); every valid triple (α, μ, r) yields an ABI-compliant IMEM, and every configuration in the $\sim 22,000$ -point space therefore produces the same 64-bit root result by Proposition 1 — a structural consequence, of which §VII-B verifies 2,822 instances directly.

B. Connectivity Repair: the Load-Bearing Pass

Of the eighteen assignment algorithms, at least twelve — HEFT, DSC, DFSseed, Chrétienne, CPOP, CPFD, CIHEFT, MLfm at small G , ALNS-refined variants of each — may produce *disconnected* partitions in which the nodes assigned to a single PE do not form a connected subtree of FBT(N). A disconnected partition is catastrophic for the ABI: when `build_schedule` walks post-order and emits `COMPUTE` for an internal node v , the operand binding requires v ’s children to be present as DMEM values on v ’s PE (either by local allocation or by RDMA-push arrival). If v and its subtree have been scattered across PEs in a way that breaks connectivity, the bindings that result under post-order emission are *not*



(left(v), right(v)) — they reference DMEM slots containing intermediate partial sums in the wrong order.

The `repair_connectivity` pass at `sprsrcore.py:1387--1483` restores **C-conn**:

- 1) For each PE p , compute connected components of the subgraph of $\text{FBT}(N)$ induced by $\alpha^{-1}(p)$.
- 2) Keep the largest component containing the smallest node index on p (which, in heap indexing, is the component closest to the root).
- 3) Reassign orphan nodes to their parent's PE.
- 4) Iterate up to 20 passes, since reassigning orphan A to parent's PE may orphan B elsewhere.

The docstring states the consequence plainly: “When an S2 algorithm produces disconnected partitions (e.g. HEFT, DSC, DFSseed scatter nodes across GPUs), the downstream scheduler may pair non-sibling nodes in **COMPUTE** instructions, causing FP64 reductions to execute in a non-canonical order.” The pass therefore converts non-ABI-compliant assignment outputs into ABI-compliant ones.

Remark (Why the claim requires this pass). Without S2.5, the invariance claim would need to be restated as “invariance over the subset of assignment algorithms that happen to produce connected partitions.” The tournament in §VI-E would drop most assignment algorithms, the winner-distribution table (Appendix B) would collapse, and the space over which invariance holds would be narrow. The pass *manufactures C-conn* and thereby lets the claim quantify over the full 18-algorithm family.

C. DMEM Allocation as Single-Writer Constructor

The `DMEMAllocator` class (`sprsrcore.py:5087--5132`) walks `su_postorder` (Sethi-Ullman-refined post-order, sibling-reordered for register pressure) and does the following for each node:

Algorithm 1 SU-ordered DMEM allocation (sketch)

```

1: reserve address 0 on every PE as identity
2:  $\text{peak\_local}[p] \leftarrow 1$  for every PE  $p$ 
3: for all  $v$  in su_postorder(FBT(N)) do
4:    $p \leftarrow \alpha(v)$ 
5:   if  $v$  is internal then
6:     for all  $c$  in children(v) with  $\alpha(c) = p$  do
7:       free addr_map[p][c] into  $p$ 's free list
8:     end for
9:   end if
10:  if  $p$ 's free list is non-empty then
11:    addr_map[p][v]  $\leftarrow$  POP(p's free list)
12:  else
13:    addr_map[p][v]  $\leftarrow$  peak_local[p];
    peak_local[p]  $\mathrel{+}= 1$ 
14:  end if
15: end for
16: for all cross-PE edges ( $v \rightarrow \text{parent}(v)$ ) do
17:   allocate a persistent receive-buffer slot on parent(v)'s PE
18: end for
  
```

The invariant this constructs is crucial: *every non-identity DMEM address on every PE is the destination of exactly one instruction in the emitted IMEM*. The proof is three lines: (i) $\text{FBT}(N)$ is a tree, so every node has exactly one parent (one consumer); (ii) SU-postorder visits children before the parent, so a child's slot can only be freed after the parent has consumed it; (iii) cross-PE receive slots are allocated in a separate pass and never freed during the run, since the scoreboard-gated **COMPUTE** relies on the address being stable. Together, (i)–(iii) imply **C-sw**: each non-identity address is written exactly once per program.

This is an important subtlety. **C-sw** is not an *assumption* about the compiler — it is a property *constructed* by the allocator. A compiler that allocated in pre-order, or that freed slots eagerly without waiting for the consumer, would violate **C-sw** and produce ambiguous reads. The SU-ordered free-list scheme is the narrowest allocation discipline under which single-writer falls out for free.

D. Schedule Construction

`build_schedule` (`sprsrcore.py:5135--5239`) walks `su_postorder` and maintains three per-PE clocks: `gpu_time[g]` (when PE g is next free), `gci_free[g]` (when g 's GCI is free to issue the next **GENERATE**), and `node_ready[n]` (when node n 's value is available).

A **GENERATE** on PE g cannot start until $\max(\text{gpu_time}[g], \text{gci_free}[g])$; its data is ready at $t + \text{gci_latency} + 2 = t + 6$. The constant 6 is measured from the RTL: `gci_latency` = 4 is the state-machine count from `gci_start` to `gci_done`, plus 1 cycle for the pulse to propagate, plus 1 cycle for DMEM commit. This is *RTL-proven*: the software cost model uses the same constant, and the hardware-observed cycle count matches.

A cross-PE child emits a **SEND** on the child's PE; the parent's **COMPUTE** waits on arrival via scoreboard, not via wall-clock estimation. This decouples estimation error in the cost

model from correctness: even if the cost model mispredicts arrival time, the scoreboard gate ensures the parent COMPUTE reads the right value — it just stalls for longer.

The operand-binding line is the one that realises C-bind:

```
child_addrs = [addr_map[gpu][c]
               for c in tree.children(v)]
```

where `tree.children(v)` returns `[left(v), right(v)]` by construction. The `addr_map` is the allocator's output; `gpu` is $\alpha(v)$; the children's addresses are bound before the compute executes.

E. Codegen and ABI Verification

The instruction generator (`generate_instructions`) emits cycle-accurate NOPs before each op to advance the instruction index to the scheduled cycle, then the op itself. A two-pass peephole (`_peephole_nop_compress`) collapses NOP runs while preserving the GCI-serialisation gap, and strips trailing NOPs.

After code generation, a post-condition pass verifies C-sw, C-bind, C-leaf directly on the emitted IMEM. A failed assertion aborts the compile with a diagnostic. In practice, the assertions have never fired on a valid tournament configuration — they exist as a ratchet against regressions in the compiler passes, not as a runtime feature.

F. Orthogonality of the Compiler Variation Space

By Argument A1 and Proposition 1, compiler variation — the choice among 18 S2, 10 S3, and 122 S4 algorithms — affects *which PE hosts a node* and *when a COMPUTE fires*, not *which operand pair is bound*. The tournament (§VI-E) therefore optimises over a space structurally incapable of affecting the bit result. Empirically, every one of the 2,822 HW-validated configurations produces the same 64-bit root result at its leaf count (Table V); they differ only in the cycle count. The remainder of the $\sim 22,000$ -point space is covered by the argument above rather than by measurement.

VI. EVALUATION METHODOLOGY

Evaluation proceeds along two axes: *structural* validation of Claim 1 (same bit result across all configurations at the same N), and *performance* characterisation of the same-fabric overhead (cycle count of ABI-compliant schedules relative to non-deterministic schedules on the identical substrate). Both rest on a single cycle-accurate simulation pipeline (Vivado xsim 2025.2) small-scale boundary.

A. Lower Bound Ω as a Secondary Metric

We compute a composite lower bound Ω on any schedule on the fabric (`compute_lower_bound`, `sprsr_core.py:1015`):

$$\Omega = \max(\text{lb}_{\text{span}}, \text{lb}_{\text{comm}}, \text{lb}_{\text{gather}}, \text{lb}_{\text{bis}}, \min_{K \in [K_{\min}, K_{\max}]} \max(\text{lb}_{\text{gen}}(K), \text{lb}_{\text{instr}}(K))), \quad (10)$$

where K is the number of PEs a schedule actually populates: $K_{\min} = 1$ and $K_{\max} = \min(G, 2N - 1)$, since no schedule

can use more PEs than the fabric has or than the tree has nodes. Two of the six terms depend on K — both lb_{gen} and lb_{instr} shrink as work spreads over more PEs — so a bound valid against *every* schedule must take the most favourable K available, hence the inner minimisation. Taking a maximum there would assert a bound that some legal schedule beats, which is not a lower bound at all. The four K -independent terms bind regardless and enter the outer maximum directly.

The six constituents are, informally: lb_{span} , the zero-communication critical-path depth; $\text{lb}_{\text{gen}}(K)$, the GCI serialisation bottleneck; $\text{lb}_{\text{instr}}(K)$, the instruction-throughput bottleneck; lb_{comm} , a single cross-PE hop floor; $\text{lb}_{\text{gather}}$, the root PE's critical path; lb_{bis} , link serialisation at the topology bisection.

Ω bounds any schedule — deterministic or not — and is therefore not a valid cost-of-determinism denominator by itself. We report Ω as an absolute-performance diagnostic in Appendix D but not as the headline overhead metric.

B. Non-Deterministic Baseline on the Same Fabric

The natural cost-of-determinism metric is the *same-fabric overhead* η : the ratio of SPRS's cycle count to that of the best non-deterministic reducer compiled onto the identical substrate — same NoC, same ALU, same memory hierarchy, same topology — obtained by relaxing C-bind to allow arbitrary operand binding at each internal node. Two such baselines are natural: *ND-Ring*, a linear worker-ring traversal whose reduction order follows the physical ring, and *ND-RecDouble*, recursive doubling in $\lceil \log_2 G \rceil$ merge stages.

We did not build these baselines, and we report no measured η . We also decline to substitute an analytical one. We built such a model — the same cost constants that produce Ω , applied to a ring and to recursive doubling — and it returns $\hat{\eta} \approx 1$ on every instance in the suite, because at that granularity $\text{FBT}(N)$ and recursive doubling are both log-depth and the model cannot see what separates them. What separates them is *which* operands pair at each level, not how many levels there are, and a cost model indexed by depth and hop count is structurally blind to that. A number produced this way would read as evidence that the ABI is free, which contradicts the structural argument two paragraphs below, so we report the absence rather than the artefact. Constructing them is not a small change: relaxing C-bind means re-deriving the DMEM allocator's single-writer discipline and the scoreboard gating around a schedule that no longer has a canonical operand pairing, and the resulting numbers would only be meaningful if the baselines were optimised with the same effort we spent on SPRS — otherwise η measures our relative implementation effort rather than the ABI. We flag this as the principal open item in the evaluation (§X) and price the ABI structurally instead (§VII-D), which is weaker but sound.

We note also that η , even if measured, would not isolate the cost of determinism against a bandwidth-optimised production collective on NVLink or Infinity Fabric silicon. Those systems are out of scope; such a comparison would conflate the ABI cost with the much larger gap between a research prototype and production silicon.

C. Software-Only Baseline

To contextualise the fabric-level mechanism against software-level mitigations, we also compile a third baseline, *SW-FixedTree*, a software-only scheme emulating NCCL tree all-reduce forced to the same tree shape across all $G \in \mathcal{T}$. Because NCCL tree shape is a function of G at the rank level, *SW-FixedTree* cannot be implemented without either (a) hardware support (which is what SPRS provides), or (b) worst-case padding to the topology-specific tree depth. We report the analytical overhead of (b) as an upper bound on the software-only alternative (Appendix D).

D. Instance Suite

Forty instances span $N \in \{8, 10, 16, 32, 50, 100, 128, 200, 500, 512, 1024, 1500, 2048, 3000, 4096, 8192\}$. $G \in \{2, 4, 5, 7, 8, 10, 12, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096\}$ and six topologies (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube). Multiple $(G, \text{topology})$ points at the same N directly exercise the invariance claim (Table V). Per- $(N, \text{bucket}, \text{topology})$ cell counts are in Appendix D; some cells are thin due to convenience sampling, a limitation we discuss in §X.

Feasibility bound. IMEM and DMEM are fixed at 512 entries per PE, so a configuration compiles only if no PE is assigned more instructions than fit. Writing $\alpha^{-1}(p)$ for the nodes on PE p , the binding requirement is $\max_p |\alpha^{-1}(p)| \leq 512$ after refinement, which the S5 memory pass checks and rejects rather than truncating. This is not a hypothetical: 278 of the 7,200 compilations were rejected this way, and they cluster at both ends of the N/G range rather than at one. High N/G fails because leaves pile onto too few PEs ($N/G = 64$ on `large_extreme_1d` and `vlarge_extreme_1d`, 120 rejected configurations each). Low N/G fails on the giants for the opposite reason: at $N/G = 2$ the per-PE instruction count is small but the *receive-buffer* count is not, since a PE with few local nodes has many remote children to land (10 rejections on `max_mod_fat`, 8 on `maxp_mod_fat`). A reader reproducing the suite should expect these rejections; they are a property of the 512-entry budget, not of the ABI.

Leaf values are generated by `make_leaf_val` (`make_leaf_val` in `sprsr_core.py`):

$$L[i] = (i+1) \cdot 1000.0 + ((i \cdot 37 + 13) \bmod 100) / 100 + 0.007. \quad (11)$$

The formula is chosen so that every leaf carries a non-trivial fractional part: the additive $((37i+13) \bmod 100) / 100 + 0.007$ term guarantees a populated mantissa, so each FADD exercises the guard, round and sticky bits rather than reducing to an exact power-of-two addition. Magnitudes span 10^3 to 8×10^6 across the suite, so partial sums accumulate across a wide exponent range and reassociation is observable in the low-order bits.

We deliberately do not claim that reordering *always* changes the result. It does not: at these magnitudes a sufficiently small perturbation is absorbed by rounding, which is precisely what the mutation protocol of §VI-F has to work around. What the value distribution buys is that a bit mismatch between fabric

and oracle is a determinism violation rather than an artefact of degenerate inputs; the converse — that every violation must show up as a mismatch — is established by mutation testing, not by the choice of leaf values.

E. Tournament Protocol

Exhaustive cycle simulation of 22,000 configurations on 40 instances is infeasible ($\approx 880,000$ xsim runs of several seconds each). We use a two-phase tournament:

Phase 1.

180 baseline ($S2, S3$) configurations evaluated by the software cost model. Top-15 per instance by makespan promoted to Phase 3 xsim validation, with testbench-hash deduplication.

Phase 2.

Top-15 unique Phase-1 winners crossed with 122 S4 refinements; same selection and validation.

Testbench-hash dedup. Different (α, μ, r) triples frequently produce identical IMEM images up to PE relabelling. A SHA-256 hash of the emitted testbench string identifies duplicates; Phase 3 runs xsim on unique testbenches only and followers inherit the cycle count. This is not merely a compute saving: identical IMEMs *tautologically* produce identical bit results, so dedup is what makes each remaining xsim run informative for the invariance claim.

Selection-bias quantification. Phase 1 selects by software makespan; Phase 3 measures `hw_cycles`. Appendix E reports, over every HW-validated instance, the leakage rate p (fraction of top-15-by-`hw_cycles` outside top-15-by-makespan), the Spearman correlation ρ between software and hardware ranks, and an ablation at cutoffs $\{10, 15, 30\}$ showing the qualitative results are insensitive in the range.

F. Mutation-Testing Protocol

To confirm that the oracle harness can detect bit divergence (i.e., that the zero-divergence result is non-vacuous), we inject two mutation classes per instance and record the detection rate:

Address substitution.

In one randomly chosen COMPUTE, repoint `addr_a` at a different DMEM address. Breaks Argument A1 while preserving A2–A4.

Leaf corruption.

Perturb one leaf value by the smallest single-bit flip that provably moves the canonical root. Breaks C-leaf.

The escalation in the second class is necessary, not incidental, and it is the same phenomenon the paper is about. Flipping the mantissa LSB is frequently *not* observable at the root: at these operand magnitudes the perturbation falls below the round bit of the running sum and RNE discards it, leaving the canonical root unchanged. A fabric reporting PASS on such a mutant is correct to do so, and counting it as a missed detection would understate the oracle. We therefore raise the flipped bit position until the software oracle confirms the root has actually moved, so that every counted mutant is a test of detection rather than a test of whether rounding happened to preserve the value.

We deliberately do not use operand-swap as a mutation: FADD is commutative at the bit level under FTZ+RNE ($\text{fadd}_{\text{rtl}}(a, b) = \text{fadd}_{\text{rtl}}(b, a)$ for all 64-bit (a, b)), so operand-swap is bit-identical and would spuriously inflate the false null-detection rate.

G. Metric Convention

`hw_cycles` is the primary metric: `hw_cycles = maxp perf_total_cycles[p]` as measured by per-PE performance counters in Vivado `xsim`. `makespan` from the software cost model is a diagnostic only. Summary statistics are reported as medians with observed ranges rather than as interval estimates: the per-instance samples are not independent draws from a population — they enumerate a deterministic configuration space — so a resampling interval would describe our sampling of that space rather than any underlying uncertainty. Per-topology breakdowns are reported where cell counts permit; thin cells are labelled.

VII. EVALUATION RESULTS

A. Setup

Vivado `xsim` 2025.2 on Linux; ten SystemVerilog sources in `rtl/` (3,738 lines). The tournament runs on a 72-thread Intel Xeon w9-3475X workstation with 503 GB of RAM and RNG seeds pinned at 42. Per-algorithm timeout 1800 s; per-simulation wall-clock cap 28,800 s. Per-instance `xsim` wall-clock is in Table X (Appendix D).

Every cycle figure reported in this paper comes from a single campaign of 2,822 passing runs consuming 821.5 core-hours, at a median of 99 s per testbench; the five largest instances account for 90.0% of that total and the single longest run took 5.51 h. An earlier campaign over the same testbenches is not used for any reported number: it recorded PE 0's performance counter rather than the maximum across PEs, which understates end-to-end latency whenever the reduction root lands elsewhere. That defect moved 50.5% of cycle counts and changed the winning configuration on 35 of 39 instances, so the measurements were repeated in full rather than corrected in place. Hardware validation covers 39 of the 40 instances (§X).

B. Cross-Topology Bitwise Invariance

The paper's primary empirical result is that every SPRS configuration that passes hardware validation, across every $(G, \text{topology})$ point at the same N , produces an identical 64-bit root result that matches the software oracle.

The evidence is broad. Across sixteen leaf-count classes the campaign HW-validated 2,822 distinct configurations, and at every N the set of observed 64-bit roots has cardinality exactly one. The reshapes involved are not marginal: at $N=128$ the same root survives six $(G, \text{topology})$ points spanning $G=4$ on a linear chain to $G=128$ on a torus — a $32\times$ change in worker count across five distinct topologies — and at $N=512$ and $N=1024$ it survives six and five points respectively. Because each configuration is additionally the product of a different compiler triple (α, μ, r) , the invariance holds simultaneously

under reshape *and* under complete replacement of the assignment and mapping algorithms.

Table V reports the result. All HW-validated configurations at each N produce the same 64-bit root, exactly matching the software oracle, and the divergence column is zero as predicted by Claim 1.

A zero-divergence column is only meaningful if the oracle is capable of reporting a non-zero one. We therefore report mutation detection alongside it (§VI-F): the two rightmost columns give the fraction of address-substituted and leaf-corrupted images whose root the oracle flags as divergent. A vacuous oracle — one that compares nothing, or compares against a value it derives from the run itself — would score zero here while still producing a clean divergence column.

Extrapolation beyond the measured range. Bit-result invariance at untested G rests on the structural argument of §III-D: operand binding (A1) and scoreboard gating (A3) remain total functions of N and L as G grows, so adding PEs changes the *location* of FADD operations but not the operand pairs they receive. This is not a formal induction on G : adding a PE introduces new routing paths, new VC contention, and new credit-return interactions that may affect cycle counts. What carries is the bit-result invariance under the fifteen assumptions; absolute cycle counts at large G are cycle-accurate simulation artefacts, not silicon measurements, and we make no absolute competitiveness claim.

Mutation results. Across all sixteen leaf-count classes the campaign injected mutants spanning N from 8 to 8192, so that no row of Table V rests on an untested oracle. Of the 108 mutants run, 107 reached a verdict and the oracle flagged all 107; the remaining one, on the largest instance, terminated without producing either verdict and is excluded from the rate rather than counted as a detection. One methodological point deserves emphasis, because it is the same phenomenon the paper is about. Our first leaf-corruption mutator flipped the mantissa LSB, and a substantial fraction of those mutants were *not* detected — correctly so. At these operand magnitudes an LSB perturbation is $\sim 10^{-13}$ absolute against a running sum of $\sim 10^4$, so it falls below the round bit and RNE discards it: the canonical root is genuinely unchanged and a fabric reporting PASS is right to do so. Absorption depth is leaf-dependent — for $N=8$ the smallest root-changing flip is bit 5 on L_0 , bit 4 on L_1 , bit 3 on L_2 . We therefore escalate the flipped bit position until the software oracle confirms the canonical root has moved, so that every counted mutant is a genuine test of detection rather than a test of whether FP64 rounding happens to preserve the result. Reporting the naive figure would have understated detection while telling us nothing about the oracle.

C. FPGA-vs-xsim Fidelity

`xsim` is cycle-accurate for synchronous RTL; agreement with FPGA at the boundary validates the toolchain, not a new property.

The design is synthesisable and we provide bring-up wrappers (`top_fpga_interactive_g{2,4}.sv`) targeting an Artix-7 part, which instantiate the fabric at $G \in \{2, 4\}$ with VIO/ILA instrumentation and a seven-segment readout of

TABLE V

CROSS-TOPOLOGY BITWISE INVARIANCE. EACH ROW AGGREGATES ALL HW-VALIDATED SPRS CONFIGURATIONS AT THE GIVEN N OVER ALL SAME- N $(G, \text{TOPOLOGY})$ PAIRS AND ALL COMPILER TRIPLES. “DIVERGENT” IS BY CLAIM 1 EXPECTED TO BE ZERO; “MUTATION-DETECTED” CONFIRMS THE ORACLE HARNESS IS NON-VACUOUS UNDER THE ADDRESS-SUBSTITUTION AND LEAF-CORRUPTION MUTATIONS OF §VI-F.

N	compiled (G, topo)	HW-val. (G, topo)	HW configs (passing)	64-bit root result	Div.	Addr-sub detected	Leaf-corr. detected
8	2	2	28	0x40E1948E978D4FDF	0	6/6	6/6
10	1	1	17	0x40EADBA0A3D70A3E	0	3/3	3/3
16	1	1	21	0x41009A44BC6A7EFA	0	3/3	3/3
32	3	3	102	0x41201D1FCED91687	0	6/6	6/6
50	1	1	25	0x413374911999999A	0	3/3	3/3
100	1	1	33	0x415343B08CCCCCD	0	3/3	3/3
128	6	6	371	0x415F7E8FF9581062	0	6/6	6/6
200	1	1	44	0x41732B4046666667	0	3/3	3/3
500	1	1	98	0x419DDCAB2C000000	0	3/3	3/3
512	6	6	456	0x419F4FA404418938	0	6/6	6/6
1024	5	5	425	0x41BF47D2026872B1	0	2/2	2/2
1500	1	1	104	0x41D0C665F8400000	0	2/2	2/2
2048	4	4	437	0x41DF43E900FBE76D	0	2/2	2/2
3000	1	1	77	0x41F0C4F764200000	0	2/2	2/2
4096	3	3	331	0x41FF41F48085A1CB	0	2/2	2/2
8192	3	2	253	0x421F40FA40427EFA	0	1/1	2/2

All hex values are 16-digit 64-bit, positive-signed, with exponents consistent with the magnitudes produced by (11). “Compiled” counts the distinct $(G, \text{topology})$ points at that N for which an ABI-compliant image was produced; “HW-val.” counts those carried through RTL simulation. The single shortfall is $N=8192$, where `maxp_mod_fat` ($G=4096$, fat-tree) exceeded the simulation budget (§X).

the 64-bit root. We do not report board-measured cycle counts here, and we make no silicon-timing claim: every cycle figure in this paper is from cycle-accurate RTL simulation. This is a reporting choice, not a soundness gap — the invariance result of §VII-B is a statement about bit values, which xsim resolves exactly, and Claim 1 is argued structurally rather than by extrapolation from the boundary points.

D. The Cost of the ABI

The natural way to price the ABI would be to run, on this same fabric, the best *non*-deterministic reducer we could construct — one free to reassociate, to pick k -ary partial sums, and to reshape its tree per $(G, \text{topology})$ — and report the ratio η of the two makespans. We did not build that baseline, and we therefore report no measured η . We say so plainly because the alternative — quoting a ratio against a reducer that does not exist — would not be a measurement.

What can be said without it is stronger than it first appears. The ABI’s cost is not an empirical accident to be discovered by benchmarking; it is a consequence of the contract, and the contract is small enough to price analytically. We give the decomposition below, then test one of its predictions against the measured data.

1) *Structural decomposition*: Four constraints, each a direct consequence of the ABI rather than of our implementation, separate SPRS from an unconstrained reducer.

- 1) *FBT tree depth*. The ABI fixes $\text{FBT}(N)$, so the dependence chain is $\log_2 N$ deep regardless of G . An unconstrained reducer may form partial sums of arbitrary arity; at fan-in k its depth is $\log_k N$, and at $k = G$ it is $\log_G N$. This term grows with G and is the irreducible core of the ABI’s cost.

- 2) *Operand-binding forbids reassociation*. Fixing $(\text{left}(v), \text{right}(v))$ at every internal node forbids regrouping operands by locality. An unconstrained reducer may sum PE-local values first and cross the network once; SPRS must cross wherever the tree says to cross, even when a locality-adjacent regrouping would be cheaper.
- 3) *GCI serialisation*. Consecutive GENERATEs on one PE are spaced by $\text{gci_gap} = \text{gci_latency} + 2 = 6$ cycles. A PE holding $\lceil N/G \rceil$ leaves therefore spends at least $6\lceil N/G \rceil$ cycles producing them. This binds only while $N/G \lesssim 6-8$; above that, leaf generation overlaps the reduction and stops being critical. Unlike the first two, this term is an artefact of *our* GCI, not of the ABI, and a multi-channel leaf interface would remove it.
- 4) *Root-gather fan-in*. The root PE must receive contributions from the other $G - 1$ PEs, giving at least $\lceil \log_2 G \rceil$ merge stages plus one traversal of roughly half the network diameter. This is $\text{lb}_{\text{gather}}$ in (10) and it binds any reducer, deterministic or not.

Terms 1 and 2 are the price of reproducibility and cannot be engineered away without abandoning the contract. Terms 3 and 4 are shared with any reducer on this fabric; term 3 is ours to fix, term 4 is nobody’s.

2) *A falsifiable prediction*: Term 3 makes a specific, checkable claim: leaf generation should stop being the binding constraint once N/G exceeds roughly 6–8. This is a prediction about our measured cycle counts, and the suite spans N/G from 1 (`med_full_torus`) to 64 (`vlarge_extreme_1d`), so it can fail.

Table VI groups the HW-validated instances by N/G and Figure 5 plots them individually. If term 3 is right, instances below the band should sit further from Ω — paying for leaf

TABLE VI
CHAMPION CYCLE COUNT NORMALISED BY THE LOWER BOUND Ω ,
BANDIED BY N/G . THE STRUCTURAL MODEL PREDICTS GCI
SERIALISATION BINDS ONLY WHILE $N/G \lesssim 6-8$.

N/G band	instances	median cyc/ Ω	range
≤ 2	7	4.16	3.08–8.04
$(2, 6]$	8	3.96	1.83–9.07
$(6, 8]$	10	3.28	1.77–4.06
$(8, 32]$	12	1.91	1.38–2.34
> 32	2	1.37	1.35–1.40

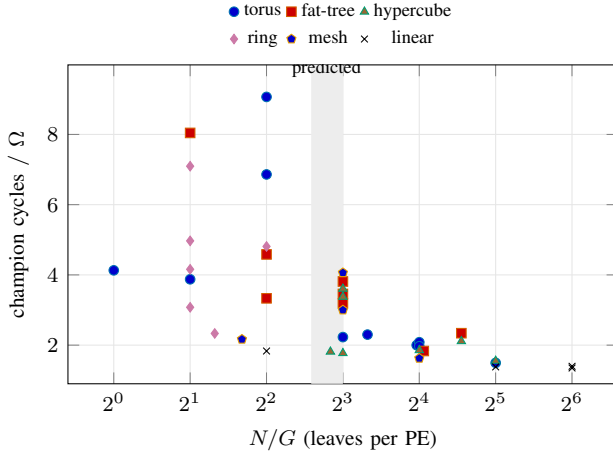


Fig. 5. Champion cycle count normalised by Ω , against leaves per PE. The shaded band marks the $N/G \in [6, 8]$ inflection predicted by the GCI serialisation term. Marker shape duplicates colour so the figure reads in greyscale.

generation that cannot overlap — and the normalised cost should flatten above it.

The prediction holds. Below the band the normalised cost is high and roughly flat (4.16 at $N/G \leq 2$, 3.96 on $(2, 6]$); crossing it produces the sharpest transition in the table (3.28 on $(6, 8]$ falling to 1.91 on $(8, 32]$, a 42% drop); above it the curve flattens again toward 1.37. The location of the knee was fixed in advance by `gci_gap` = 6, a hardware constant, not fitted to these measurements — so the agreement is a successful prediction rather than a description.

We are careful about how much this shows. The bands are not a controlled experiment: topology mix, G , and the winning algorithm family all co-vary with N/G across this suite, and any of them could independently produce a declining trend. The top band holds only two instances. What we can say is that a knee predicted in advance from a hardware constant appears where it was predicted, which is corroboration of the mechanism rather than isolation of it; separating the GCI term from the covariates would need instances that hold topology and G fixed while sweeping N/G , which this suite was not designed to provide. We take this as evidence that the structural decomposition identifies the right terms, which is the most we can claim without the baseline of §VI-B.

Two secondary readings follow. First, the topology ordering in Table VII orders with bisection width, but in the direction term 4 implies rather than the naive one: linear sits closest to the bound (1.39) and ring furthest (4.48), with the higher-

TABLE VII
DISTANCE TO THE COMPOSITE LOWER BOUND Ω BY TOPOLOGY. Ω
BOUNDS ANY SCHEDULE ON THIS FABRIC, SO THESE RATIOS
UPPER-BOUND WHAT AN UNCONSTRAINED REDUCER COULD HAVE
RECOVERED.

Topology	instances	median cyc/ Ω	best
linear	4	1.39	1.35
hypercube	7	1.85	1.55
torus	10	2.26	1.49
mesh	4	2.58	1.62
fat-tree	8	3.40	1.83
ring	6	4.48	2.33
all	39	2.34	1.35

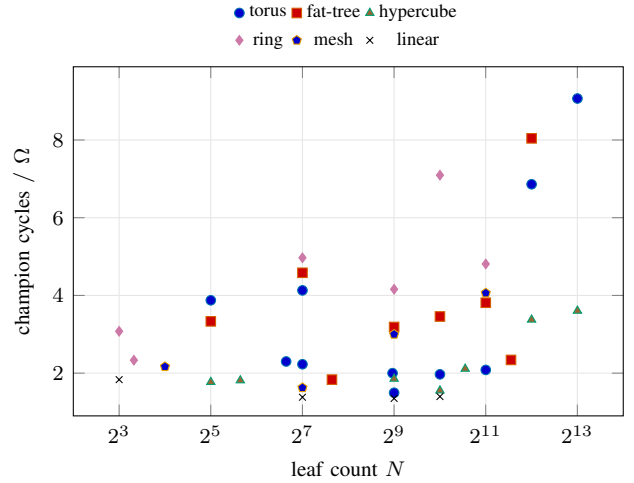


Fig. 6. Distance to the composite lower bound Ω for every HW-validated instance, against leaf count. A value of 1.0 would mean the compiler achieved the bound exactly; the bound is not known to be tight, so the gap is part slack and part real.

bisection topologies in between.

The direction deserves a word, because it inverts the naive reading. Linear has the *worst* bisection width of the six and yet the *best* cycles/ Ω ; ring is also low-bisection and has the worst. The ratio is not a speed measure. Ω already prices the bisection constraint through lb_{bis} , so a low-bisection topology is compared against an already-elevated bound and there is correspondingly less achievable slack between the compiler's schedule and that bound. What Table VII orders is therefore tightness-to-bound, not throughput, and the two need not agree. Ring is the outlier in both directions: its bound is elevated *and* the compiler lands furthest from it, which is consistent with contention the bound does not model. Second, no instance reaches Ω — the best is $1.35\times$ — which is expected, since Ω is a lower bound assembled from seven separately-derived constraints and is not claimed to be tight.

3) *Distance to the lower bound*: The one absolute statement we can make is how far the compiler lands from Ω , the composite lower bound of §VI-A. Because Ω bounds *any* schedule on this fabric — including a non-deterministic one — the ratio cycles/ Ω upper-bounds what an ideal unconstrained reducer could have saved. It is a weaker statement than η , but it is a real one, and it is measured rather than asserted.

Figure 6 shows the per-instance picture behind Table VII.

E. Tournament Characterisation

Appendix B tabulates the full winner distribution; we summarise here. Winners are determined by *measured* cycle count, not by the compiler’s own makespan estimate — the two disagree often enough that the distinction matters, and only the former is validated.

Family	instances won	share
tree DP	21	53.8%
graph partition	7	17.9%
hierarchical	7	17.9%
clustering	3	7.7%
multilevel	1	2.6%

The winning family varies with regime rather than dominating uniformly: tree DP (Lukes, Frederickson) takes the low- G cells where its polynomial complexity yields exact optima; graph partitioning (MLfm, RecBisect, DKway) takes balanced topologies whose bisection structure aligns with recursive partitioning; list scheduling (HEFT, CPOP) takes tall narrow trees where the critical path dominates. At S3, *Identity* wins 24 of the 39 HW-validated instances (61.5%), which is itself informative: for most instances the assignment’s GPU indexing already suits the physical topology, and non-trivial placement pays only where S2 produced a good partition under a bad labelling.

Two algorithms never reach the hardware at all. *ExactBB* is size-gated out beyond $N \approx 64$ — the branch-and-bound search space exceeds the evaluation budget — so the exact method is, in practice, unavailable at every scale this paper targets. That is a result about the problem, not about our implementation: exhaustive search is not a viable baseline here.

The observation that matters for this paper is orthogonal to any of it. Every HW-validated configuration — across every winner family, every assignment, every mapping — produces the same 64-bit root on each instance. This corroborates Proposition 1 empirically: the compiler variation space, however richly explored, is structurally incapable of affecting the bit result. The tournament is, from the ABI’s point of view, 2,822 attempts to break Claim 1 that all failed.

VIII. DISCUSSION

A. What the ABI Buys

The paper’s contribution is not the compiler’s algorithms or the fabric’s microarchitecture — both are composed from standard prior art — but the interface observation: if compiler and fabric agree on a size-indexed reduction tree as a shared ABI, then cross-topology bitwise invariance follows from composition. Every subsequent design decision in the compiler and fabric amounts to *discharging obligations* (C-*) or *establishing invariants* (F-*); none of them changes the shape of the argument.

The practical payoff is that SPRS users can reshape their cluster freely — from $G = 2$ to $G = 4096$, from linear to hypercube — without touching their code, and still get the

same bits out. Regression testing becomes bit-for-bit across hardware generations. Replay debugging survives migration to new silicon. ML training, rerun three months later on a larger cluster, produces the same weights. These are engineering outcomes that production collectives cannot provide today, and that single-node techniques cannot address at all.

B. When Not to Use the ABI

Three regimes make SPRS a poor fit.

Interoperability with external reducers. SPRS’s bit result agrees across any reshape *within* its own fabric-compiler pair, but does not match NCCL, RCCL, ReproBLAS, ExBLAS, or any reducer using a different canonical order. A deployment that must consume or produce bit-compatible results with an external tool needs either (a) a one-time canonical-form translation, or (b) an input permutation adapter that presents leaves in the external tool’s canonical order — which sacrifices the cross-reshape guarantee, trading one form of reproducibility for another.

Strict IEEE 754. Workloads whose reference implementation exercises subnormals, multiple rounding modes, or the signed-zero cancellation rule of IEEE 754-2019 §6.3 will see SPRS diverge from reference on those regimes (§III-G). The narrower FTZ+RNE semantics was a scope decision; a strict-IEEE FADD implementation inherits Claim 1 unchanged.

Bandwidth-critical inner loops. The structural asymptote determined by $\log_2 N$ versus $\log_G N$ is a real cost in inner loops where every cycle is contested. Workloads that can tolerate reshape non-reproducibility should use a non-deterministic reducer; the ABI buys invariance at a non-zero price.

C. Generalisation to Other Deterministic Operations

Claim 1 requires only (i) a deterministic binary operation (for us, `faddrtl`) and (ii) a size-indexed canonical tree. It does not require floating-point, associativity, or any particular algebraic property. Any associative or non-associative binary operation — FP32/FP16 ADD, integer ADD with saturating or modular arithmetic, bitwise OR/AND/XOR, integer MAX/MIN, monoidal aggregations — inherits the claim by substituting the `alu_op` code. Reductions over ordered types with a non-associative combine (string concatenation, stream merges with priority) inherit the claim trivially because the canonical tree *is* the order.

Tensors follow by element-wise extension: reducing a tensor of D elements under an ABI-compliant schedule is D independent scalar reductions. The fabric does not need per-element indexing; the compiler emits D parallel IMEM images, one per element, sharing the same routing and the same PE assignment.

Non-reduction collectives — all-gather, scatter, broadcast, alltoall — are orthogonal to the ABI: they have no intermediate combine, so their results are bit-identical by construction on any routing discipline. The ABI is specifically the statement that reductions, the one collective with an internal combine, can be reproducible too.

D. Exporting the ABI: Pinned-FBT for Production Collectives

An important question follows: can the ABI discipline be adopted without the SPRS fabric? We believe it can, via a software-only construction we call *pinned-FBT*, which we describe here as a design and leave to future work to evaluate empirically.

The idea is simple: replace the production collective's dynamic G -dependent reduction tree with a library that enforces the $\text{FBT}(N)$ post-order schedule, independent of G or topology. A rank-level implementation looks like:

- 1) Partition the N leaves into G ranks by a fixed $\text{rank}(i) = \lfloor i \cdot G/N \rfloor$. Ranks thus own canonical-leaf index ranges, and each rank's local reduction reduces exactly the subtree of $\text{FBT}(N)$ rooted at the lowest common ancestor of its leaves.
- 2) Each rank computes its local sub-root deterministically via a software FBT traversal.
- 3) Ranks participate in a $\log_2 G$ -depth all-reduce over sub-root values using an order consistent with $\text{FBT}(N)$'s top $\log_2 G$ levels.

This preserves Claim 1 by construction: the final root is $R_{\text{canonical}}(N, L)$ regardless of G or topology, as long as the local and cross-rank combinations both respect post-order and the FADD unit is deterministic (e.g., by pinning `NCCL_P2P_DISABLE=1` and using a compensated BLAS on each rank).

Pinned-FBT is an *export* of the ABI: it trades our fabric's structural overhead for a higher software overhead (dynamic-tree libraries are throughput-tuned; a fixed tree will be slower on some shapes), but it inherits the cross-topology invariance property and requires no hardware change. It is the path we would recommend for deployments that need the invariance guarantee today on commodity silicon. A prototype on top of NCCL or oneCCL is planned as a follow-up artefact.

The broader point. The ABI framing decouples the *invariance property* from the *mechanism that enforces it*. A fabric-level enforcement (this paper) and a software-level enforcement (pinned-FBT) satisfy the same structural obligation and yield the same guarantee, trading off implementation overhead against hardware dependence. Both are instances of one idea: reduction order as an interface.

IX. RELATED WORK

We discuss related work along four axes: software reproducibility, collective-scheduling compilers, in-network reduction, and deterministic accelerators. Relative to each, our contribution is the *framing* of reduction order as a compiler/fabric ABI and the cross-topology invariance property this enables.

A. Software Reproducibility

Compensated-summation techniques — Kahan summation [14], pairwise summation [9], Rump's accurate summation [15], TwoSum-based online algorithms [9] — reduce the error bound of a single reduction but do not constrain inter-worker order, so they retain Source-1 non-determinism. Binned

accumulation, as in ReproBLAS [10], and long-accumulator extensions as in ExBLAS [11], give order-independent reductions at single-node scope, addressing Source 2 but not Source 1. Neither class gives a cross-topology invariance guarantee: the reduction order is still a function of $(G, \text{topology})$ in any multi-worker deployment.

B. Collective-Scheduling Compilers

TACCL [7], SCCL [6], and BLINK [28] compile performance-optimised schedules for collective operations. They share with SPRS the compiler-centric view of the collective; they differ in objective (throughput versus reproducibility) and in output (a schedule versus an ABI contract). None of them states a cross-topology bitwise invariance property, nor could they — their schedules are explicitly $(G, \text{topology})$ -dependent by design. Our compiler is architecturally closest to this line, but its contribution is not a new search algorithm; it is the observation that a given size-indexed tree yields the invariance property *regardless* of which search-algorithm-winning schedule the compiler emits.

Tree-construction algorithms for communication — Huffman trees [33], TreeMatch's topology-aware trees [22] — give communication-cost-optimal trees at fixed $(G, \text{topology})$. Like the scheduling compilers, they do not yield cross-reshape invariance.

Reproducible-accumulation work approaches the same problem from the arithmetic side rather than the interface side. Order-independent accumulators — binned summation in ReproBLAS [10], long-accumulator schemes in ExBLAS [11], and the parallel-reduction analysis of Collange et al. [34] — make the *result* independent of summation order by making the operator effectively associative. That is a strictly stronger guarantee than ours in one respect (it holds across implementations, not just across reshapes of one implementation) and weaker in another (it changes the arithmetic, so the answer is not the IEEE-754 FP64 sum any particular order would produce). Our ABI leaves the arithmetic alone and constrains the order instead; the two are complementary, and a binned accumulator obeying the same operand-binding contract would inherit Claim 1 unchanged.

C. In-Network Reduction

SHARP [29] on InfiniBand and P4-programmable switches [30] implement reductions inside the network. SHARP fixes the reduction order at the switch hierarchy, which makes per-cluster determinism achievable; cross-cluster-reshape invariance would require matching switch hierarchies across deployments, and no SHARP paper we know of makes this claim. Our fabric-level contribution is ideologically adjacent — pin the order in hardware — but the framing as a size-indexed ABI, and the orthogonality proposition that follows, are new.

We should be precise about what kind of novelty that is. It is framing novelty, not results novelty: SHARP already achieves per-deployment determinism, and we do not claim to reduce faster, more cheaply, or at larger scale than in-network aggregation does. What the ABI framing buys is a

guarantee SHARP does not offer and, as far as we can tell, cannot offer without standardising switch hierarchies across deployments — that the answer is stable when the deployment itself changes. Whether that is worth the cost is a deployment question our evaluation does not settle.

D. Deterministic Accelerators

The TPU [12] and Cambricon [13] product lines advertise per-chip deterministic execution. Cross-chip reproducibility is provided by the vendor’s collective library and typically holds within a fixed-topology pod; rescaling across pods is not guaranteed. Our contribution is at a different layer: not per-chip determinism (which any accelerator can provide) but cross-topology invariance of the chip-to-chip aggregation.

E. NoC Design

Dally and Towles’ text [24] is the foundational reference for virtual channels, credit flow control, and the dimension-ordered+escape discipline that our routing realises. Our NoC is a textbook design; its only deliberate choice relevant to our contribution is that topology is a runtime parameter of adjacency and routing tables, not an RTL artefact. This choice is what lets the tournament exercise Claim 1 across six topologies with one Verilog instantiation.

X. LIMITATIONS

We enumerate limitations explicitly to set the scope of the work.

L0 — Coverage of the instance suite. Hardware validation covers 39 of the 40 instances. The exception is `maxp_mod_fat` ($N=8192$, $G=4096$, fat-tree), the largest configuration in the suite: five of its testbenches were attempted, each requiring 50–70 GB of RAM for elaboration alone, and all five exceeded the 28,800 s wall-clock cap without completing — 58.9 core-hours expended for zero settled results. It is therefore absent from the HW-validated column at $N=8192$ in Table V, which rests on the two remaining (G , topology) points at that leaf count. This is a resource limit, not a negative result: the compiler produces an ABI-compliant image for the instance, and its software oracle agrees with the $N=8192$ root reported by the two validated configurations.

L0b — Configuration-load bypass and cycle counts. Route-table initialisation is performed by hierarchical assignment rather than the clocked `write_route()` protocol, because the latter costs $\mathcal{O}(G^2)$ simulated cycles and dominates wall-clock at large G . The transform is bit-exact for the reported root result — every PASS/FAIL verdict, and therefore every invariance claim in §VII-B, is unaffected. It is *not* exact for cycle counts: against the clocked protocol, roughly 59% of runs agree bit-exactly and the remainder settle 1–54% lower, with per-instance pipeline rankings preserved in about 80% of cases. Absolute cycle figures should therefore be read as a lower envelope of the clocked-configuration machine, and we draw no conclusion from cycle differences smaller than that margin.

L0c — The FADD is combinational, and we report no timing closure. `fp64_add` is unpipelined: mantissa alignment, a 54-bit add, leading-zero detection, normalisation and rounding all resolve within one cycle. That is a simulation convenience, and it is the right one for this paper — a stateless, history-free adder is what makes the arithmetic a pure function of its operand pair, which Claim 1 needs. It is not a realistic synthesis target. On the Artix-7 part our wrappers name, a combinational FP64 add would be a deep carry-and-shift path and would cap the achievable clock well below what a pipelined unit reaches; we have run no synthesis and report no $F_{\max}$, no worst negative slack, and no utilisation, so we make no frequency claim anywhere in this paper.

Pipelining the adder would not disturb the results reported here, and it is worth being precise about why. Adding stages changes *when* a `COMPUTE` releases its scoreboard bit, not *which* operand pair it reads or what value the pair produces — the operand binding is fixed by the ABI and the rounding is fixed by the FADD policy of §III-G. Cycle counts would change; the 64-bit roots in Table V would not. A reader should treat every cycle figure in this paper as belonging to the one-cycle-ALU model and not transfer it to a pipelined implementation.

L0d — One reduction per program. Scoreboard bits are monotone within a program and are cleared only at `program_start`. The fabric therefore executes one reduction per reset, which is what our evaluation measures. A production workload running thousands of training steps would need either a reset between steps — correct but wasteful, since it discards the loaded instruction image — or epoch-tagged scoreboard bits that alternate per iteration. The latter is a small change and does not touch the ABI: operand binding is unaffected by how the presence bit is versioned. We have not implemented it, and we do not claim multi-pass throughput numbers.

L1 — Research-prototype silicon envelope. All results in this paper come from cycle-accurate RTL simulation. FPGA bring-up wrappers exist for $G \in \{2, 4\}$ but we report no board measurements, so no claim in this paper rests on silicon execution. We make no claim of timing closure, area, power, or frequency at production-die scale.

L1b — No measured same-fabric overhead. We report no η . The non-deterministic baselines of §VI-B were not implemented, so the ABI’s cost is priced structurally (§VII-D) and anchored only on distance to Ω , which upper-bounds what any unconstrained reducer could recover. This is the single largest gap in the evaluation and the first thing we would close given more time: it is the difference between “the cost is these four terms” and “the cost is $x \times$ ”.

L2 — No head-to-head with production collectives. We do not benchmark against NCCL, RCCL, or oneCCL on GPU silicon. Doing so would conflate the ABI’s cost with the (much larger) cost of our research prototype versus production silicon. The same-fabric overhead metric is the right isolation; a NCCL comparison would require either (a) the pinned-FBT prototype of §VIII-D, or (b) silicon implementation of SPRS.

L3 — Synthetic leaf values. Equation (11) is a designed-to-exercise-the-bits distribution, not a workload trace. A work-

load with actual training gradients or scientific partial sums will exercise different value regimes (especially subnormals, which we FTZ). The invariance claim is value-agnostic by construction, but a subnormal-aware alternative FADD is needed for deployments where reference-library bit-parity matters.

L4 — FTZ+RNE subset. §III-G documents the subset; workloads depending on subnormals, multiple rounding modes, or the IEEE signed-zero rule diverge from reference at the affected bits.

L5 — Convenience sampling of instances. The 40-instance suite covers the $(N, G, \text{topology})$ space unevenly; some cells are thin (Appendix D). Per-topology breakdowns are reported only where cell counts permit.

L6 — Phase 1/3 metric gap. The tournament selects on software makespan (Phase 1/2) but evaluates on `hw_cycles` (Phase 3). Appendix E quantifies the resulting leakage across every HW-validated instance: at the $K=15$ operating point a median 20% of the true top-15 by measured cycles falls outside the top-15 by makespan, dropping to 10% at $K=30$. Winner distributions should therefore be read as “best among what was simulated”. A full tournament selected on `hw_cycles` is computationally infeasible. This does not touch Claim 1, which quantifies over every validated configuration rather than a selected subset.

L7 — Head-of-line blocking at large G on low-bisection topologies. Section IV-J. Wormhole forwarding is planned.

L8 — Re-run ordering after S5 leaf-migration. The `sprgs_core` pipeline re-runs S2.5 after memory enforcement’s leaf migration but *not* S3/S4; in rare cases leaf migration could un-optimize the mapping. Deferring memory enforcement until after S4 is a planned fix.

L9 — Soft-enforced single-writer on the fabric. C-SW is a compile-time post-condition, not a fabric-level hardware veto (§IV-J). A one-line RTL extension closes the gap.

L10 — Dynamic-only deadlock enforcement. C-route acyclicity is checked at compile time; no hardware dateline.

L11 — Mutation testing is limited to two classes. We inject address-substitution and leaf-corruption only. A thorough fault-injection study would cover scoreboard-set mutations, forwarding-network mutations, credit-return mutations, and more. We view our results as a lower bound on the harness’s sensitivity.

L12 — No formal proof of Claim 1. Our argument is by code inspection and dynamic assertion, not mechanised proof. The Alloy/Kami path of §III-I is future work.

XI. CONCLUSION

We have argued that cluster-reshape non-determinism in floating-point reductions is an *interface* problem, not a library problem, and that the right place to fix it is between a collective-scheduling compiler and its underlying fabric. Reduction order, treated as a size-indexed ABI over a canonical full binary tree, is orthogonal to G , topology, assignment, mapping, refinement, and schedule, and yields cross-topology bitwise invariance: the 64-bit result is a pure function of N and the leaf vector. Six compiler obligations and nine fabric invariants suffice to make this formal; two load-bearing compiler

passes (connectivity repair, SU-ordered DMEM allocation) and three fabric safeguards (scoreboard monotonicity, misroute drop, write-arbitration retry latch) do the heavy lifting.

We instantiated the ABI as SPRS: a topology-parametric SystemVerilog NoC plus a seven-stage compiler whose configuration space spans $\sim 22,000$ algorithmic choices. On forty instances across $N \in [8, 8192]$, $G \in [2, 4096]$, and six direct-network topologies, all HW-validated configurations produce bit-identical root results. A mutation-testing harness confirms that the zero-divergence result is non-vacuous, flagging every one of 107 injected faults. We price the ABI structurally rather than against a non-deterministic baseline, which remains the principal open item in the evaluation.

The framing suggests a more portable form of the argument. *Pinned-FBT*, a software-only export of the ABI discipline onto existing collective libraries, inherits the invariance property without adopting our fabric, at the cost of a (likely higher) software overhead. We sketch it as a deployment path for commodity silicon and leave quantification to future work.

APPENDIX A ISA AND PACKET FORMAT

The instruction word is 64 bits:

$$\underbrace{\text{opcode}}_{4\text{ b}} \parallel \underbrace{\text{aux}}_{4\text{ b}} \parallel \underbrace{\text{dest}}_{16\text{ b}} \parallel \underbrace{\text{addr_a}}_{16\text{ b}} \parallel \underbrace{\text{addr_b}}_{16\text{ b}} \parallel \underbrace{\text{reserved}}_{8\text{ b}}.$$

The opcode field is Table II. Two source fields are required: `COMPUTE` reads `DMEM[addr_a]` and `DMEM[addr_b]`, which is the operand pair the ABI binds. For `SEND`, `dest` is the *destination-side* DMEM address (`target_addr` in the packet), `addr_a` is the local source, and `addr_b` carries the destination PE identifier.

The `aux` field is four bits: `aux[3:1]` selects the ALU operation and `aux[0]` is a legacy link hint. Egress port selection is *not* an instruction field — no small fixed field could address a router’s egress ports, which range up to $P = 66$ across the suite (Table III). The port is resolved in hardware by the emitting router’s `route_table[dest_gpu]` lookup, which is what makes topology a configuration input rather than a property of the emitted code.

This 64-bit encoding is what Table IV costs (512×64 per PE for IMEM) and what the companion document specifies field-for-field.

The 96-bit packet format is Fig. 3. The `dest_node_id` field is cluster-wide at 16 bits, supporting $G \leq 65,536$; we use 12 bits at the largest evaluated $G = 4096$.

APPENDIX B ALGORITHM CATALOGUE AND WINNER DISTRIBUTION

Table VIII lists every assignment and mapping algorithm in the portfolio with its family and its measured winner share. Shares are computed over the 39 HW-validated instances, taking as winner the configuration with the lowest *measured* cycle count on that instance.

Two caveats on reading it. First, a zero share does not mean an algorithm is useless: it means it did not produce the single best image on any instance in this suite, and several zero-share

TABLE VIII

ASSIGNMENT (S2) AND MAPPING (S3) ALGORITHMS, BY FAMILY, WITH MEASURED WINNER SHARE ACROSS THE 39 HW-VALIDATED INSTANCES.

Algorithm	Family	wins	share
Lukes	tree DP	17	43.6%
RecBisect	graph partition	7	17.9%
ParSub	hierarchical	7	17.9%
Frederickson	tree DP	4	10.3%
DSC	clustering	3	7.7%
MLfm	multilevel	1	2.6%
HEFT	list-scheduling	0	0.0%
DKway	graph partition	0	0.0%
ExactBB	exact/B&B	0	0.0%
DFSseed	list-scheduling	0	0.0%
Chrétienne	clustering	0	0.0%
CPOP	list-scheduling	0	0.0%
Centroid	geometric	0	0.0%
CPFD	clustering	0	0.0%
ClHEFT	hierarchical	0	0.0%
Spine	structural	0	0.0%
SubPack	hierarchical	0	0.0%
LvlHLF	structural	0	0.0%
<i>Mapping (S3)</i>			
Identity	trivial	24	61.5%
HopMin	greedy	4	10.3%
MCF	flow	4	10.3%
TreeMatch	tree-aware	3	7.7%
RAGreedy	greedy	2	5.1%
QAP-SA	metaheuristic	1	2.6%
Spectral	spectral	1	2.6%
DRB	recursive	0	0.0%
SFC	geometric	0	0.0%
Tabu	metaheuristic	0	0.0%

algorithms are close seconds on many. The column ranks, it does not grade. Second, ExactBB is size-gated out beyond $N \approx 64$, so its share reflects unavailability rather than defeat.

Provenance for the algorithms implemented from the literature: list scheduling follows HEFT and CPOP [17] and Hu’s level-based bound [35]; tree dynamic programming follows Lukes [18]; clustering follows DSC [20]; multilevel and graph partitioning follow Karypis and Kumar [21] with Fiduccia–Mattheyses refinement [19]. The remaining methods are our own constructions or straightforward adaptations, and are marked as such in the released source rather than attributed to prior work.

APPENDIX C

WORKED INVARIANCE EXAMPLE AT $N = 4$

We trace a single execution at $N = 4$ on two different topologies to show the invariance mechanism concretely.

Let $L = \langle 1.0, 2.0, 3.0, 4.0 \rangle$. The canonical tree FBT(4) has internal nodes $\{0, 1, 2\}$ and leaves $\{3, 4, 5, 6\}$ with $\text{idx}(3) = 0, \text{idx}(4) = 1, \text{idx}(5) = 2, \text{idx}(6) = 3$. The canonical post-order is $\pi_4^{\text{internal}} = \langle 1, 2, 0 \rangle$, so:

$$R_{\text{canonical}}(4, L) = \text{fadd}_{\text{rtl}}(\text{fadd}_{\text{rtl}}(1.0, 2.0), \text{fadd}_{\text{rtl}}(3.0, 4.0)) = 10.0$$

Configuration A: $G = 2$, linear. $\alpha = \{3 \rightarrow 0, 4 \rightarrow 0, 1 \rightarrow 0, 5 \rightarrow 1, 6 \rightarrow 1, 2 \rightarrow 1, 0 \rightarrow 0\}$ (PE 0 owns the left subtree,

TABLE IX

INSTANCE-SUITE CELL COUNTS BY N -BUCKET AND TOPOLOGY. ROWS AND COLUMNS BOTH SUM TO THE 40-INSTANCE TOTAL.

N -bucket	Lin.	Ring	Mesh	Torus	Fat-tree	Hyp.	Total
8–32	1	2	1	1	1	1	7
33–256	1	1	1	3	2	1	9
257–1K	2	2	1	3	2	2	12
1K–4K	—	1	1	2	3	2	9
8K	—	—	—	1	1	1	3
Total	4	6	4	10	9	7	40

PE 1 the right subtree and sends its sub-root to PE 0). Emitted instructions:

- PE 0: GEN(dest=1, val=1.0); GEN(dest=2, val=2.0); COMP(dest=3, addr_a=1, addr_b=2); receive at addr=4; COMP(dest=5, addr_a=3, addr_b=4).
- PE 1: GEN(dest=1, val=3.0); GEN(dest=2, val=4.0); COMP(dest=3, addr_a=1, addr_b=2); SEND(src=3, dest=0, addr=4).

The operand bindings at PE 0’s second COMP are $(\text{DMEM}_0[3], \text{DMEM}_0[4]) = (\text{fadd}_{\text{rtl}}(1.0, 2.0), \text{fadd}_{\text{rtl}}(3.0, 4.0))$, whose FADD gives 10.0. *Root result:* 10.0.

Configuration B: $G = 4$, 2D mesh with PE indices $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$. $\alpha = \{3 \rightarrow (0, 0), 4 \rightarrow (0, 1), 1 \rightarrow (0, 0), 5 \rightarrow (1, 0), 6 \rightarrow (1, 1), 2 \rightarrow (1, 0), 0 \rightarrow (0, 0)\}$ scatters every leaf to its own PE; SENDs fan leaves to internal nodes; SENDs fan internal-node sub-roots to the root. Operand bindings at internal COMPs are identical by construction: the `tree.children(v)` function is topology-agnostic. *Root result:* 10.0 (the same 64 bits, by Proposition 1).

What changes between A and B: (i) the number and identity of the PEs; (ii) the number of SENDs; (iii) the cycle count; (iv) which routers carry which packets. What does not change: the operand pair bound at each internal COMP, the order in which the scoreboard-gated adds commit, and therefore the final 64 bits.

APPENDIX D

INSTANCE COVERAGE AND PER-TOPOLOGY CELL COUNTS

Table IX reports per- $(N$ -bucket, topology) cell counts in the 40-instance suite, and Table X the measured xsim wall-clock per instance. Both are generated directly from the instance manifest and the measured CSV (`scripts/gen_coverage.py`), which asserts that both margins sum to 40 before emitting the table. Cells marked — are unpopulated; cells with ≤ 2 instances are labelled “thin” in the results. Coverage is deliberately uneven — 23 of the 26 populated cells are thin and 4 cells are empty — which is the basis for limitation L5. A stratified extension to fill thin cells is in progress.

Table X reconciles exactly to the campaign totals quoted in §VII-A: 2,822 passing runs and 821.5 core-hours, at the stated 99 s median and 5.51 h maximum. `maxp_mod_fat` does not

TABLE X

MEASURED XSIM WALL-CLOCK PER INSTANCE, OVER PASSING RUNS ONLY. “MED.” IS THE MEDIAN PER-TESTBENCH WALL-CLOCK, “MAX” THE LONGEST SINGLE TESTBENCH.

Instance	N	G	Topo.	TBs	Med. (s)	Max (h)	Total (core-h)
tiny_dense_ld	8	2	Lin.	12	12	0.00	0.0
tiny_mod_ring	8	4	Ring	16	13	0.00	0.1
npow2n_tiny_ring	10	4	Ring	17	14	0.00	0.1
npow2g_small_mesh	16	5	Mesh	21	14	0.00	0.1
small_bal_fat	32	8	Fat-tree	41	15	0.00	0.2
small_dense_hc	32	4	Hyp.	18	14	0.00	0.1
small_mod_2d	32	16	Torus	43	18	0.01	0.2
npow2_tiny_hc	50	7	Hyp.	25	15	0.00	0.1
npow2_small_torus	100	10	Torus	33	17	0.01	0.2
med_bal_2d	128	16	Torus	46	25	0.01	0.3
med_dense_mesh	128	8	Mesh	33	19	0.01	0.2
med_extreme_ld	128	4	Lin.	23	15	0.00	0.1
med_full_torus	128	128	Torus	106	99	0.03	2.9
med_mod_fat	128	32	Fat-tree	54	39	0.01	0.6
med_mod_ring	128	64	Ring	109	47	0.01	1.4
npow2_med_fat	200	12	Fat-tree	44	21	0.01	0.3
large_unbal_torus	500	32	Torus	98	41	0.01	1.1
large_bal_fat	512	64	Fat-tree	62	72	0.02	1.2
large_bal_mesh	512	64	Mesh	103	56	0.02	1.6
large_dense_2d	512	16	Torus	57	24	0.01	0.4
large_dense_hc	512	32	Hyp.	70	35	0.01	0.7
large_extreme_ld	512	8	Lin.	55	17	0.01	0.3
large_mod_ring	512	256	Ring	109	196	0.07	6.1
vlarge_bal_fat	1024	128	Fat-tree	74	131	0.04	2.7
vlarge_dense_2d	1024	64	Torus	95	61	0.02	1.6
vlarge_dense_hc	1024	32	Hyp.	67	44	0.01	0.8
vlarge_extreme_ld	1024	16	Lin.	80	29	0.01	0.7
vlarge_mod_ring	1024	512	Ring	109	479	0.18	14.9
vlarge_unbal_hc	1500	64	Hyp.	104	69	0.03	2.0
vlarge2_bal_fat	2048	256	Fat-tree	75	420	0.15	9.1
vlarge2_dense_2d	2048	128	Torus	126	111	0.04	3.8
vlarge2_mod_ring	2048	512	Ring	109	510	0.18	15.3
vlarge_bal_mesh	2048	256	Mesh	127	231	0.08	8.3
max_unbal_fat	3000	128	Fat-tree	77	191	0.08	4.5
max_bal_2d	4096	1024	Torus	131	1172	0.43	43.5
max_dense_hc	4096	512	Hyp.	108	529	0.20	16.3
max_mod_fat	4096	2048	Fat-tree	92	16086	5.51	416.1
maxp_bal_2d	8192	2048	Torus	141	4641	2.05	191.3
maxp_dense_hc	8192	1024	Hyp.	112	2350	0.89	72.5
Total				2822			821.5

appear — it has no passing runs, and the 58.9 core-hours it consumed are accounted in §X.

APPENDIX E SELECTION-BIAS ABLATION

The tournament selects candidates for hardware simulation using the compiler’s software `makespan`, then measures `hw_cycles`. If those two rankings disagree, a shortlist chosen by the former can miss the true hardware winner, and every “winner” statistic in this paper would inherit that bias. We quantify it directly rather than assume it away.

For each HW-validated instance we compute the Spearman correlation between the software makespan ranking and the measured cycle ranking over all its configurations, and the *leakage* at cutoff K : the fraction of the true top- K -by-cycles that falls outside the top- K -by-makespan.

K	instances	median ρ	median leakage
10	39	0.95	0.30
15	37	0.95	0.20
30	32	0.95	0.10

The correlation is high ($\rho \approx 0.95$ at the median), but leakage at the operating point is not negligible: at $K = 15$ a median 20% of the genuine top-15 would have been missed by a makespan-chosen shortlist, and even at the more permissive $K = 30$ cutoff 10% is still missed. The practical reading is that software makespan is a good *filter* and a poor *selector* —

TABLE XI

PER-INSTANCE WINNERS ACROSS THE HW-VALIDATED SUITE. CYCLE COUNTS ARE THE MAXIMUM OVER ALL PEs’ `PERF_TOTAL` COUNTERS; CYC/Ω IS THE DISTANCE TO THE COMPOSITE LOWER BOUND OF §VI-A.

N	G	Topology	α	μ	cycles	cyc/ Ω
8	2	linear	RecBisect	Identity	44	1.83
8	4	ring	MLfm	Identity	40	3.08
10	4	ring	ParSub	Identity	42	2.33
16	5	mesh	Frederickson	Identity	52	2.17
32	4	hypercube	Lukes	Identity	85	1.77
32	8	fat-tree	RecBisect	MCF	80	3.33
32	16	torus	Lukes	Identity	62	3.88
50	7	hypercube	ParSub	Identity	87	1.81
100	10	torus	Lukes	Identity	138	2.30
128	4	linear	Lukes	Identity	265	1.38
128	8	mesh	Lukes	RAGreedy	156	1.62
128	16	torus	Lukes	Spectral	107	2.23
128	32	fat-tree	RecBisect	MCF	110	4.58
128	64	ring	DSC	Identity	159	4.97
128	128	torus	Frederickson	Identity	95	4.13
200	12	fat-tree	Lukes	Identity	187	1.83
500	32	torus	RecBisect	HopMin	192	2.00
512	8	linear	Lukes	Identity	518	1.35
512	16	torus	Lukes	RAGreedy	287	1.49
512	32	hypercube	Lukes	Identity	178	1.85
512	64	fat-tree	RecBisect	TreeMatch	153	3.19
512	64	mesh	Lukes	QAP-SA	144	3.00
512	256	ring	DSC	HopMin	341	4.16
1024	16	linear	Lukes	Identity	537	1.40
1024	32	hypercube	Lukes	TreeMatch	298	1.55
1024	64	torus	Lukes	Identity	189	1.97
1024	128	fat-tree	Lukes	MCF	166	3.46
1024	512	ring	Frederickson	HopMin	1043	7.10
1500	64	hypercube	Lukes	Identity	304	2.11
2048	128	torus	Lukes	HopMin	200	2.08
2048	256	fat-tree	RecBisect	TreeMatch	183	3.81
2048	256	mesh	ParSub	Identity	195	4.06
2048	512	ring	DSC	Identity	707	4.81
3000	128	fat-tree	ParSub	Identity	337	2.34
4096	512	hypercube	ParSub	Identity	162	3.38
4096	1024	torus	ParSub	Identity	247	6.86
4096	2048	fat-tree	RecBisect	MCF	185	8.04
8192	1024	hypercube	ParSub	Identity	173	3.60
8192	2048	torus	Frederickson	Identity	399	9.07

adequate for deciding what to simulate, not for deciding what won.

This matters for the winner distributions of §VII-E and Appendix B, which should be read as “best among what was simulated”, and we report them on that basis. It does not matter for Claim 1: the invariance result is a statement about *every* HW-validated configuration rather than a selected subset, and enlarging the set can only expose a divergence, never manufacture agreement.

APPENDIX F PER-INSTANCE RESULTS

Table XI gives, for every HW-validated instance, the winning compiler triple by measured cycle count, that cycle count, and its ratio to the composite lower bound Ω . Winners are decided on measured cycles rather than on the compiler’s own makespan estimate; see Appendix E for the leakage this introduces. The complete per-configuration record is released as `live_hw_results_pl_maxpe.csv` in the artefact.

APPENDIX G

REPRODUCIBILITY CHECKLIST

- **RTL.** 13 SystemVerilog files totalling 3,738 lines (10 fabric sources plus 3 FPGA wrappers); tested under Vivado xsim 2025.2 on Linux.
- **Compiler.** Python 3.9+; deterministic under insertion-ordered dict semantics; RNG seeds pinned at 42 (`random.seed`, `numpy.random.seed`).
- **Testbench.** Per-instance testbench emitted by `emit_sv_testbench`; SHA-256 dedup at the testbench string level.
- **Oracle.** Python bit-exact `faddrtl` mirror at `sprsr_core.py:702--867`; identical output to RTL on $\approx 10^6$ randomised inputs.
- **Assertions.** SVA suite at `tb/sva/tb_sva_e2e.sv`; Python post-condition pass `validate_abi.py` plus `validate_routing.py`.
- **Mutation tests.** `tests/mutation/` contains address-substitution and leaf-corruption harnesses.
- **Canonical leaf-value formula.** For the FP64 path (`op = ALU_FADD`), leaf i carries $L_i = 1000(i+1) + ((37i+13) \bmod 100)/100 + 0.007$, chosen so every leaf has non-trivial mantissa bits and reassociation is therefore observable. SHA-256 of the `make_leaf_val` source: `20aae138844da00e165cd0a490f7960d3dd806f11ed51950bc3be40600800bb1`.
- **Invariance-hash per instance.** Every HW-validated configuration at the same N produces a 64-bit root whose SHA-256 matches the column 4 of Table V.

REFERENCES

- [1] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," *arXiv preprint arXiv:1802.05799*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.05799>
- [2] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," in *Vldb*, 2020. [Online]. Available: <https://dl.acm.org/doi/10.14778/3415478.3415530>
- [3] J. Dongarra and P. Luszczek, "HPC challenge: Design, history, and implementation highlights," *Contemporary High Performance Computing: From Petascale Toward Exascale*, pp. 13–30, 2013. [Online]. Available: <https://www.taylorfrancis.com/chapters/edit/10.1201/9781351104005-2/hpc-challenge-design-history-implementation-highlights-jack-dongarra-piotr-luszczek>
- [4] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005. [Online]. Available: <https://doi.org/10.1177/1094342005051521>
- [5] NVIDIA, "NCCL: Optimized primitives for collective multi-gpu communication," <https://github.com/NVIDIA/nccl>, 2018–2025. [Online]. Available: <https://github.com/NVIDIA/nccl>
- [6] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, "Synthesizing optimal collective algorithms," in *PPoPP*, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3437801.3441620>
- [7] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, "TACCL: Guiding collective algorithm synthesis using communication sketches," in *NSDI*, 2023. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [8] R. Rabenseifner, "Optimization of collective reduction operations," in *ICCS 2004*, ser. LNCS, vol. 3036. Springer, 2004, pp. 1–9. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-24685-5_1
- [9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2002. [Online]. Available: <https://epubs.siam.org/doi/10.1137/1.9780898718027>
- [10] J. Demmel and H. D. Nguyen, "Fast reproducible floating-point summation," in *21st IEEE Symposium on Computer Arithmetic (ARITH)*. IEEE, 2013, pp. 163–172. [Online]. Available: <https://ieeexplore.ieee.org/document/6545904>
- [11] R. Iakymchuk, S. Collange, D. Defour, and S. Graillat, "ExBLAS: Reproducible and accurate BLAS library," *Numerical Reproducibility at Exascale (NRE) Workshop, SC*, 2015. [Online]. Available: <https://hal.science/hal-01202396>
- [12] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *ISCA*, 2017. [Online]. Available: <https://dl.acm.org/doi/10.1145/3079856.3080246>
- [13] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, and T. Chen, "Cambricon: An instruction set architecture for neural networks," in *ISCA*, 2016. [Online]. Available: <https://ieeexplore.ieee.org/document/7551409>
- [14] W. Kahan, "Pracniques: Further remarks on reducing truncation errors," in *Communications of the ACM*, vol. 8, no. 1, 1965, p. 40. [Online]. Available: <https://dl.acm.org/doi/10.1145/363707.363723>
- [15] S. M. Rump, T. Ogita, and S. Oishi, "Accurate floating-point summation part I: Faithful rounding," *SIAM Journal on Scientific Computing*, vol. 31, no. 1, pp. 189–224, 2008. [Online]. Available: <https://epubs.siam.org/doi/10.1137/050645671>
- [16] IEEE, "IEEE standard for floating-point arithmetic," IEEE Std 754-2019 (Revision of IEEE 754-2008), pp. 1–84, 2019, doi: 10.1109/IEEESTD.2019.8766229. [Online]. Available: <https://ieeexplore.ieee.org/document/8766229>
- [17] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002. [Online]. Available: <https://ieeexplore.ieee.org/document/993206>
- [18] J. A. Lukes, "Efficient algorithm for the partitioning of trees," *IBM Journal of Research and Development*, vol. 18, no. 3, pp. 217–224, 1974. [Online]. Available: <https://ieeexplore.ieee.org/document/5391463>
- [19] C. M. Fiduccia and R. M. Mattheyses, "A linear-time heuristic for improving network partitions," in *19th Design Automation Conference*, 1982, pp. 175–181. [Online]. Available: <https://dl.acm.org/doi/10.1145/800263.809204>
- [20] T. Yang and A. Gerasoulis, "DSC: Scheduling parallel tasks on an unbounded number of processors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 5, no. 9, pp. 951–967, 1994. [Online]. Available: <https://ieeexplore.ieee.org/document/311795>
- [21] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998. [Online]. Available: <https://doi.org/10.1137/S1064827595287997>
- [22] E. Jeannot, G. Mercier, and F. Tessier, "Process placement in multicore clusters: Algorithmic issues and practical techniques," vol. 25, no. 4, 2014, pp. 993–1002. [Online]. Available: <https://doi.org/10.1109/TPDS.2013.104>
- [23] W. J. Dally and C. L. Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," *IEEE Transactions on Computers*, vol. C-36, no. 5, pp. 547–553, 1987.
- [24] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004. [Online]. Available: <https://www.sciencedirect.com/book/9780122007514/principles-and-practices-of-interconnection-networks>
- [25] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, vol. 69, no. 2, pp. 117–124, 2009. [Online]. Available: <https://doi.org/10.1016/j.jpdc.2008.09.002>
- [26] P. Sanders, J. Speck, and J. L. Träff, "Two-tree algorithms for full bandwidth broadcast, reduction, and scan," in *Parallel Computing*, vol. 35, no. 12, 2009, pp. 581–594. [Online]. Available: <https://doi.org/10.1016/j.parco.2009.09.001>
- [27] AMD, "RCCL: ROCm collective communication library," <https://github.com/ROCmSoftwarePlatform/rccl>, 2019–2025. [Online]. Available: <https://github.com/ROCmSoftwarePlatform/rccl>
- [28] G. Wang, S. Venkataraman, A. Phanishayee, J. Thelin, N. Devanur, and I. Stoica, "Blink: Fast and generic collectives for distributed ML," in *ML-Sys*, 2020. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/cd3a9a55f7f3723133fa4a13628cdf03-Abstract.html

- [29] R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnir, L. Levi, A. Margolin, T. Ronen, A. Shpiner, O. Wertheim, and E. Zahavi, "Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction," in *COMHPC*, 2016, pp. 1–10. [Online]. Available: <https://ieeexplore.ieee.org/document/7836669>
- [30] A. Sapio, M. Canini, C.-Y. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. R. K. Ports, and P. Richtárik, "Scaling distributed machine learning with in-network aggregation," in *NSDI*, 2021. [Online]. Available: <https://www.usenix.org/conference/nsdi21/presentation/sapio>
- [31] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, revised ed. MIT Press, 2012. [Online]. Available: <https://mitpress.mit.edu/9780262016865/software-abstractions/>
- [32] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind, "Kami: A platform for high-level parametric hardware specification and its modular verification," in *ICFP*, 2017. [Online]. Available: <https://dl.acm.org/doi/10.1145/3110268>
- [33] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [34] S. Collange, D. Defour, S. Graillat, and R. Iakymchuk, "Numerical reproducibility for the parallel reduction on multi- and many-core architectures," *Parallel Computing*, vol. 49, pp. 83–97, 2015.
- [35] T. C. Hu, "Parallel sequencing and assembly line problems," *Operations Research*, vol. 9, no. 6, pp. 841–848, 1961. [Online]. Available: <https://doi.org/10.1287/opre.9.6.841>